The DDPG-Based Joint Optimization of Task Offloading and Content Caching in UAV-Assisted IoV

Jingpan Bai, Jiahui Luo, Yuan Chen, Yuming Tang, Li Jin, Yan Shi, Bozhong Yang, and Houling Ji

***Abstract*—In the unmanned aerial vehicle (UAV)-assisted Internet of Vehicles (IoV) scenario, with the advantages of rapid deployment and Line-of-Sight communication, UAVs are widely adopted to provide the computation, communication and storage service for vehicles. However, existing research often overlooks the impact of content caching on task latency and UAV resource optimization in UAV-assisted IoV. Hence, this article explores the relationship between computation offloading and content caching, and formulates a joint optimization problem to reduce task delays in UAV-assisted IoV environment. Especially, each computation task is modeled as a directed acyclic graph, in which the subtasks are interdependence. Furthermore, the joint optimization problem is the nonlinear integer programming problem, and the deep deterministic policy gradient algorithm is used to solve the joint optimization problem, regarding the decision variables, available resources, and the optimal objective as the action space, the state space, and the reward function, successively. Finally, extensive simulation experiments demonstrate the algorithm's effectiveness on task processing delay in UAV-assisted IoV environments.**

***Index Terms*—Computation offloading, content caching, deep deterministic policy gradient (DDPG), mobile-edge computing (MEC), unmanned aerial vehicle (UAV)-assisted Internet of Vehicles (IoV).**

I. Introduction

With the development of the Internet of Things (IoT) and the 5th Generation Mobile Communication Technology (5G) in recent years, the various services and applications for the Internet of Vehicles (IoV) are emerging, including information recognition, autonomous driving, augmented reality, etc. [1]. With the increase in various applications, the amount of task that needs to be processed in the IoV is also growing rapidly, the most of tasks are computation-sensitivity and time-sensitivity [2]. Nevertheless, due to the constrained resources of the vehicles, the vehicles are incapable of processing the task within a limited time [3]. The mobile-edge computing (MEC) presents an effective solution to overcome this problem. When the vehicles have no enough resources to process the tasks, they can offload the corresponding task to the edge servers. In this case, the MEC can allocate enough computation resources for the tasks processed in time. Then, the computation results are returned to the corresponding vehicles. By Deploying the MEC server, the loads on vehicles and the service delay can be alleviated. However, the conventional MEC servers can not satisfy the requirements of the vehicles in scenarios with limited or insufficient resources, such as remote regions with inadequate infrastructure, the dense urban area with insufficient resources, etc. [4]. Due to the quick deployment, the low cost and the Line-of-Sight (LoS) communication, the unmanned aerial vehicles (UAVs) are widely applied to assist edge servers for serving IoV [5]. So, the concept of UAV-assisted IoV is proposed and receives more and more attention [6].

In the UAV-assisted IoV environment, the UAV can connect with multiple vehicles and is taken as resource provisioning nodes. However, in UAV-assisted IoV environments, usually UAVs do not consider the impact of cached content on the task process delay. The cached content is transmitted from the cloud or MEC server to the network nodes [7] and computation nodes [8], which then process the corresponding tasks. However, existing research has failed to consider the increase in task transmission and processing latency. To solve this problem, many studies have proposed a joint optimization strategy of computation offloading and content caching. For example, [9] uses deep reinforcement learning to explore UAV-assisted task offloading strategies. Reference [10] focuses on the joint optimization of computational offloading and data caching in UAV-assisted IoV. Other studies, such as [11] and [12], have explored similar issues but often simplify scenarios by considering only a single UAV

Received 12 January 2025; revised 29 March 2025, 16 May 2025, and 25 June 2025; accepted 10 July 2025. Date of publication 15 July 2025; date of current version 25 September 2025. This work was supported in part by the Natural Science Fund of Hubei Province, China, under Grant 2023AFB082 and Grant 2024AFB851; in part by the Foundation of Yunnan Key Laboratory of Service Computing under Grant YNSC24117; in part by the Foundation of Yunnan Key Laboratory of Unmanned Autonomous Systems under Grant 202408YB08; in part by the Open Fund of Hubei Key Laboratory of Oil and Gas Drilling and Production Engineering (Yangtze University): Application Research of Machine Learning in Shale Gas Well Fluid Accumulation Prediction and Foam Drainage Applicability Diagnosis under Grant YQZC202402; and in part by the Open Fund of Hubei Key Laboratory of Oil and Gas Drilling and Production Engineering (Yangtze University) under Grant YQZC202503. *(Corresponding authors: Houling Ji; Jingpan Bai.)*

Jingpan Bai is with the School of Computer Science, Yangtze University, Jingzhou 430023, China, also with the Hubei Key Laboratory of Oil and Gas Drilling and Production Engineering, Yangtze University, Wuhan 430100, China, also with the Yunnan Key Laboratory of Service Computing, Yunnan University of Finance and Economics, Kunming 650221, China, and also with the Yunnan Key Laboratory of Unmanned Autonomous Systems, Yunnan Minzu University, Kunming 650504, China (e-mail: jingpan89@yangtzeu.edu.cn).

Jiahui Luo, Yuan Chen, Yuming Tang, Li Jin, Yan Shi, Bozhong Yang, and Houling Ji are with the School of Computer Science, Yangtze University, Jingzhou 430023, China (e-mail: jhouling@yangtzeu.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3588858

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

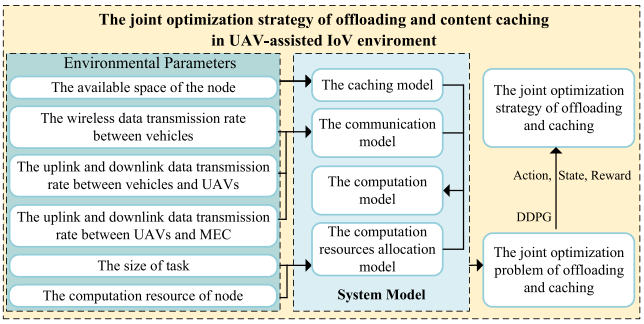


Fig. 1. Overview of research in this article.

or ignoring factors, such as UAV energy consumption and vehicle-to-vehicle (V2V) communication [13], [14], [15].

Although there have been many studies, there are still limitations in the existing studies. Many studies fail to fully consider the dynamic nature of UAV networks, the heterogeneity of computing and caching resources, or the impact of UAV energy consumption on system performance [16], [17]. In addition, some studies assume ideal conditions, such as unlimited cache capacity or binary offloading strategies, which may not affect real-world scenarios [18], [19], [20].

These limitations highlight the need for more comprehensive and practical solutions to practical problem in a UAV-assisted IoV environment. Hence, it is necessary to jointly optimize the available resource to reduce the task process delay in the UAV-assisted IoV computation environment. However, none of the above papers considered the impact of content caching on task latency and UAV resource optimization in UAV-assisted IoV. To solve this problem, referring to [21], [22], and [23], we mainly focus on the joint optimization problem of task offloading and content caching and ignore the movement of UAVs. Specifically, this article proposes a joint optimization strategy of computation offloading and content caching, using a directed acyclic graph (DAG) model to formulate the task process. Then, deep deterministic policy gradient (DDPG) is used to solve the proposed joint optimization problem, and the DDPG-based joint optimization of computation offloading and content caching (DDPG-JOTOCC) algorithm is designed. In this article, the novelty is shown as follows.

1) Each computation task is modeled as a DAG, in which the subtasks are interdependence. The DDPG joint optimization of task offloading and content caching via modeling of DAG, computation resource allocation, UAV energy consumption constraints, and content caching.
2) We explore the relationship between computation offloading and content caching, and formulate a joint optimization problem to reduce task delays in UAV-assisted IoV environment. The overview of the research is shown in Fig. 1.

The contribution of this article is summarized as follows.

1) The joint optimization problem of computation offloading and content caching is formulated to minimize the task delay in UAV-assisted IoV environment, where each computation task is modeled as a DAG and the energy constraints of UAVs are considered as well.
2) The joint optimization strategy based on DDPG algorithm is designed to solve the above the formulation problem for obtaining the optimal computation offloading and content caching decisions.
3) The extensive experiments are conducted to verify that the proposed algorithm outperforms the benchmark algorithm on the total delay of the task process, the total delay of the nonviolated tasks process, the task completion ratio (TCR), and the total vehicle energy consumption (TVEO).

The remainder of the content is organized as follows. In Section II, the related works for UAV-assisted IoV are presented. In Section III, the system model for UAV-assisted IoV is described in detail. In Section IV, the joint optimization problem is built, and the reasons that DDPG algorithm is used to solve the joint optimization problem are analyzed. The DDPG-JOTOCC algorithm is designed to achieve the optimal solution for the joint optimization problem in Section V. The use case is introduced in Section VI. Then, in Section VII, the extensive experiments are conducted, and the simulation results are analyzed. Lastly, Section VIII summarizes the content and looks forward to future work.

II. Related Works

In this section, the related works are summarized, as shown in Table I. Meanwhile, the limitation of the existed works is discussed. The detailed discussion is described as follows.

Ko et al. [8] investigated the integration of asynchronous federated learning (AFL) with blockchain technology based on DAG in edge computing environments to develop a new integrated solution, aiming to enhance the efficiency, security, and decentralization of federated learning in IoT environments. Yan et al. [9] delved into the utilization of deep reinforcement learning to design an efficient UAV-assisted vehicular edge computing task offloading strategy. Under the constraints of limited network bandwidth and limited UAV power, the trajectory and the task offloading strategy of the UAV are jointly optimized. Huang et al. [10] established a joint UAV-assisted IoV scenario, where both UAVs and macro base stations can provide computation and data caching services for smart vehicles. Then, the authors formulate a joint optimization problem for dynamic data caching and computation offloading, aiming to minimize the average task processing delay and maximize the UAV cache hit ratio. Dai et al. [11] formulated a novel online UAV-assisted vehicular task offloading problem to minimize vehicular task delay under the long-term UAV energy constraints and addressed the challenge of overloaded RSUs in urban aggregation areas in the formulated problem.

Yu et al. [12] proposed a UAV-enabled MEC system for deploying UAVs in areas with signal blockage. This facilitates MEC service provisioning for IoT devices. Experiments show the superiority of our collaborative UAV-MEC offloading scheme over baseline schemes for MEC-IoT. Wu et al. [13] presented a MEC-based framework for vehicle applications. It partitions tasks and offloads them to nodes, considering delays. The advantage is fine-grained offloading and optimization. The disadvantage is high time complexity and lack of mobility and

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ORGANIZATION OF THE EXISTING RESEARCH

Ref.	Modeling					Optimization Objectives	Application Scenarios
	Cach.	Com.	CRA.	UEL.	DAG(SSub.)		
[8]	×	×	×	×	✓	Maximizing the efficiency and security of FL in the edge computing environment	Autonomous driving
[9]	×	✓	×	✓	×	Minimizing the system delay in UAV assisted IoV environment	Smart cities and smart transportation systems
[10]	✓	✓	×	×	×	Minimizing the average task processing delay and maximizing the UAV cache hit ratio in UAV assisted IoV environment	Smart transportation systems
[11]	×	✓	×	✓	×	Minimizing the time-average vehicular task delay in UAV assisted IoV environment	Urban traffic management
[12]	×	✓	✓	✓	×	Minimizing the weighted sum of service delay and UAV energy consumption in UAV assisted MEC environment	IoT applications
[13]	×	✓	✓	×	✓	Minimizing the execution time and maximizing the throughput of applications in MEC assisted IoV environment	Autonomous driving
[14]	×	✓	×	×	×	Minimizing the total offloading cost in MEC assisted IoV environment	Smart transportation systems
[15]	×	✓	✓	×	×	Minimizing the overall latency of MEC assisted IoT system	Autonomous driving
[16]	×	✓	×	✓	×	Minimizing the average response time of all missions in UAV assisted IoT environment	Environmental monitoring
[17]	✓	×	✓	×	×	Maximizing the quality of user experience in UAV assisted IoT environment	The content distribution in the hotspot area
[18]	×	✓	×	×	×	Minimizing the average latency of user tasks and maximizing the average load fairness of UAVs in UAV assisted MEC environment	Large-Scale user scenarios
[19]	✓	✓	×	×	×	Minimizing the total network delay in MEC assisted IoV environment	Vehicle Applications in IOV
[20]	✓	✓	×	×	×	Minimizing the average execution latency for heterogeneous tasks in MEC assisted IoV environment	Smart transportation systems
[24]	×	✓	✓	✓	✓	Minimizing the average task latency and energy consumption in UAV assisted vehicular networks	Autonomous driving
[25]	×	✓	✓	×	✓	Minimizing response time of all tasks in UAV assisted MEC system	Autonomous driving
Our paper	✓	✓	✓	✓	✓	Minimizing the total task process delay in UAV assisted IoV environment	Autonomous driving

Note: Ref. stands for References. Cach. stands for Caching Decision. Com. stands for Computation Decision. CRA. stands for Computation Resources Allocation. UEL. stands for UAV Energy Limitation. SSub. stands for Serial Subtask. Serial subtasks refer to a series of tasks that are executed in sequence, with each task starting only after the previous one is completed.

network dynamics. Raza et al. [14] presented a UAV-based system for MEC services in signal-blocked areas. It optimizes UAV location, communication, and resource allocation to reduce delay and energy consumption. The disadvantage is the complexity of the optimization problem, requiring approximate algorithms. Yang et al. [15] proposed a low-latency task offloading framework based on deep reinforcement learning for computation offloading in wireless powered MEC networks. It achieves near-optimal performance with low complexity and fast adaptation. However, it assumes a binary offloading policy and nonpartitionable tasks, which may limit its applicability.

Zhu et al. [16] talked about learning-based computation offloading methods in UAVs-assisted edge computing. It has the advantages of considering complex task models, dynamic network states, and energy constraints. It has the limitation of not considering small scale fading or other uncertainties. Zhang et al. [17] optimized UAV-assisted cellular networks for user QoE. It proposes algorithms, considers UAV deployment, caching, and user association, and measures QoE with MOS. The advantage is innovative data distribution, complexity, and supply–demand balancing. The disadvantage is UAV energy, mobility, security, privacy, and limited comparisons. Tang et al. [18] studied that in UAV-assisted access edge computing networks, QoE is effectively improved through joint optimization among of task segmentation computation offloading, and computational resource allocation strategies, but UAV energy consumption constraints of UAVs in complex environments still need to be improved to achieve better system performance. Ning et al. [19] presented an edge intelligent IoV framework, which optimizes computation offloading and content caching using imitation learning, to reduce network delay and energy consumption. The advantages are that it considers the dynamics and heterogeneity of IoV, proposes an online multidecision-making scheme, and validates it with real-world traffic data. The disadvantages are that it does not consider the cooperation and communication interference among vehicles, nor compares with other AI-based methods. Li et al. [20] proposed a method to deal with heterogeneous tasks by jointly optimizing computing task offloading and edge caching. The advantage is the significant reduction in the maximum task processing time. The disadvantage is that it does not take into account the limited cache capacity of the edge server, which cannot meet the cache needs of all vehicle terminals and reduce energy consumption. Shen et al. [24] proposed a dependency task offloading method based on asynchronous federated deep reinforcement learning. By modeling task dependencies using a DAG, this method significantly reduced the average task execution latency and energy consumption in UAV-assisted vehicular networks. However, there is still

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

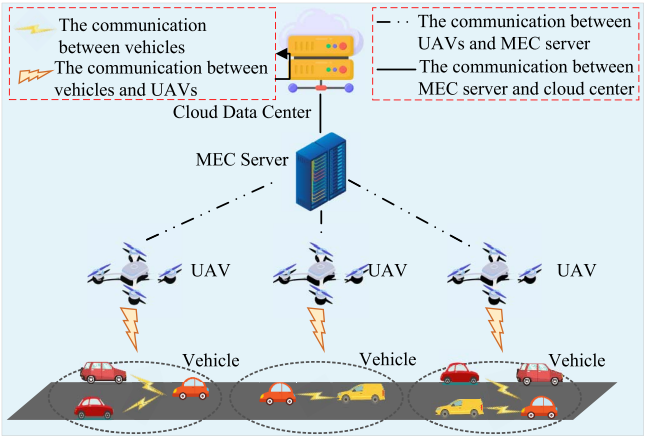


Fig. 2. UAV-assisted IoV system.

room for improvement in content caching and UAV energy consumption management. Sun et al. [25] proposed a method for scheduling computing tasks and allocating UAV resources in IoV. The advantage is that it models tasks with DAG to show dependencies and reduces the maximum task processing time. The disadvantage is that it does not take into account the energy consumption of UAVs and content caching, and thus cannot meet the caching requirements of all vehicle terminals to reduce energy consumption.

To sum up, a significant number of them failed to consider the content caching and the V2V communication. The energy consumption of UAVs was another frequently ignored factor, leading to potential oversights in the practical application of the findings. Moreover, some papers simplified their scenarios by considering only a single UAV in the environment, which could limit the generalizability of their conclusions to more complex, real-world situations.

III. System Model

The UAV-assisted IoV system consists of UAVs, vehicles, and a MEC server, as shown in Fig. 2. In this article, the MEC server is denoted by *ES*, the set of UAVs is denoted by $\mathcal{U} = \{1, \ldots, u, \ldots, U\}$. The set of vehicles is denoted by $\mathcal{V} = \{1, \ldots, v, \ldots, V\}$. Those important notations used throughout this article are summarized in Table II.

When handling real-time and computation-intensive tasks, such as the augmented reality, the high-precision map distribution [26], etc., the vehicles cannot usually finish the task process within the given time, due to the limited computation and caching resources. So, the computation and caching resources are jointly scheduled to improve the task process efficiency, in which one computation task can be divided into multiple subtasks, and these subtasks can be offloaded to different computation nodes for processing. Furthermore, each task can be modeled as a DAG $A_v = \{Y_{v,k}, E_{v,k}, \tau_v\}$ [24], [25], where $Y_{v,k} = \{y_{v,k}\}$ is the set of subtasks, $E_{v,k}$ is the dependency relationship between subtasks, and τ_v is the maximum delay a task can tolerate. $y_{v,k} = \{w_{v,k}, s_{v,k}\}$, $w_{v,k}$ represents the number of CPU maximum cycles index to process the subtask $y_{v,k}$ and $s_{v,k}$ represents the size of subtask

TABLE II
Notations Used Throughout This Article

Notation	Description
$o_{v,k}$	The cache content size for subtask $y_{v,k}$
$\mathcal{U}$	A set of UAVs
$\mathcal{V}$	A set of vehicles
A_v	The task of vehicle v
ES	The MEC server
ρ_{los}	The shadow fading component
ε	The path loss exponent
f_v	The computation resources of vehicle v
f_{uav}	The computation resources of UAV u
B_{ES}^{down}	The downlink bandwidth of MEC server
B_u^{down}	The downlink bandwidth of UAV u
B	The channel bandwidth
N_0	The noise power
H_v	The cache capacities of vehicle v
H_u	The cache capacities of UAV u
H_{ES}	The cache capacities of MEC server
d_v	The size of task A_v
$w_{v,k}$	The number of CPU clocks cycles required to execute $y_{v,k}$
p_{ES}	The sending power of MEC server
p_v	The sending power of vehicle v
p_u	The sending power of UAV u

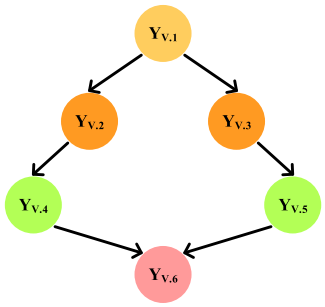


Fig. 3. Example of a DAG.

$y_{v,k}$. $k \in \mathcal{K}_v = \{1, 2, \ldots, K_v\}$ and $\mathcal{K}_v$ is the set of subtask index of task A_v. A example of a DAG is shown in Fig. 3, the tasks follow a priority order. For example, task $y_{v,6}$ can only be executed after tasks $y_{v,4}$ and $y_{v,5}$ have been completed, and the dependency relationships between subtasks of each vehicle are different. We adopt a quasi-static scenario, like [21] and [27], in which the devices and UAVs remain unmoved during a task offloading period, but may move in different periods. This means that within each time slot, the states of UAVs and vehicles are relatively stable, and the decisions on task offloading and content caching are achieved within the current time slot. In addition, we focus on the joint optimization problem of task offloading and content caching within a single time slot, without considering the trajectory action control for UAVs, like [28], [29], and [30]. The proposed algorithm can be run repeatedly to achieve the task offloading and content caching result in different time slots.

A. Caching Model

In order to improve the task process efficiency, the required content can be cached into the corresponding computation node in advance. Let $hit_{v,k,j}$ be the decision variable of content required by the subtask $y_{v,k}$ on the computation node. Specifically, $hit_{v,k,j} = 1$, if the content required by the subtask

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

$y_{v,k}$ is cached on node j. Otherwise, $hit_{v,k,j} = 0$. The content size for subtask $y_{v,k}$ is set to $o_{v,k}$, and the maximum available storage space of node j is $H_j^{\max}$. The content size cached by node cannot exceed its maximum available storage space, i.e.,

$$\sum_{v=1}^{V}\sum_{k=1}^{K_v} \mathrm{hit}_{v,k,j} o_{v,k} \le H_j^{\max} \ \forall j \in \mathcal{V} \cup \mathcal{U} \cup ES. \tag{1}$$

B. Communication Model

In this article, the communication model mainly includes the data transmission rate between vehicles, the uplink and downlink data transmission rate between vehicles and UAVs, and the uplink and downlink data transmission rate between UAVs and MEC server. From [31] and [32], we can know that there are specialized communication mechanisms between vehicles, there is no interference between them. Thus, in this article, there is no interference on the communication between vehicles. In addition, there exists interference between each vehicle and the base station on the uplink communication like [33], [34], and [35].

The data transmission rate from vehicle v to vehicle v' is denoted as

$$r_{v,v'} = B\log_2\left(1 + \frac{p_v L_{v,v'}(d_{v,v'})\kappa}{N_0}\right) \tag{2}$$

where p_v denotes the transmission power of vehicle v, N_0 denotes the noise power, κ is the vehicle path-loss constant, and B represents the channel bandwidth. $L_{v,v'}^{dB}(d_{v,v'})$ and $L_{v,v'}(d_{v,v'})$ are the path loss between vehicle v and v', $d_{v,v'}$ denotes the distance between vehicle v and v' [36], i.e.,

$$L_{v,v'}^{dB}(d_{v,v'}) = 63.3 + 17.7\log_{10}(d_{v,v'}). \tag{3}$$

The unit of (3) is dB, $L_{v,v'}(d_{v,v'}) = 10^{-[L_{v,v'}^{\mathrm{dB}}(d_{v,v'})/10]}$ is the converted equation, and the data transmission rate between vehicles after the equation integration [37], [38], [39] is

$$r_{v,v'} = B\log_2\left(1 + \frac{p_v 10^{-\frac{63.3+17.7\log_{10}(d_{v,v'})}{10}}\kappa}{N_0}\right). \tag{4}$$

One UAV can server multiple vehicles. Meanwhile, one vehicle can only communicate with one UAV [40]. We define the link state as non-LoS (*nlos*) when the communication link between two devices is blocked by the obstacles. Otherwise, it is defined as line of sight (*los*). In our manuscript, we assume that the connection between a vehicle to the UAV (V2U) is *los* [41]. Thus, the uplink data transmission rate from vehicle v to UAV u is

$$r_{v,u} = B\log_2\left(1 + \frac{p_v \rho_{\mathrm{los}} d_{v,u}^{-\varepsilon}}{\sum_{v'=1,v'\neq v}^{V} p_{v'}\rho_{\mathrm{los}} d_{v',u}^{-\varepsilon} + N_0}\right) \tag{5}$$

where $d_{v,u}$ denotes the distance between the vehicle v and the UAV u, the path loss between vehicle and UAV is $\rho_{\mathrm{los}} d_{v,u}^{-\varepsilon}$, h_u is the height of UAV u, and x_u and y_u are the coordinates of the UAV u. x_v and y_v are the coordinates of the vehicle v, ε is the path-loss exponent, ρ_{los} is the shadow fading component following the Gamma distribution. N_0 denotes the noise power, the interference of other vehicles on the vehicle v is $\sum_{v'=1,v'\neq v}^{V} p_{v'}\rho_{\mathrm{los}} d_{v,u}^{-\varepsilon}$ [42].

Furthermore, the downlink data transmission rate, i.e., the downlink data transmission rate from UAV u to vehicle v, is

$$r_{v,u}^{\mathrm{down}} = B\log_2\left(1 + \frac{p_u \rho_{\mathrm{los}} d_{v,u}^{-\varepsilon}}{N_0}\right) \tag{6}$$

$$d_{v,u} = \sqrt{h_u^2 + (x_u - x_v)^2 + (y_u - y_v)^2} \tag{7}$$

where p_u denotes the transmission power of UAV u.

Similarly, the uplink data transmission rate between UAVs and MEC server, i.e., the data transmission rate from the UAV u to MEC server is denoted by

$$r_{u,ES} = B\log_2\left(1 + \frac{p_u \rho_{\mathrm{los}} d_{u,ES}^{-\varepsilon}}{\sum_{u'=1,u'\neq u}^{U} p_{u'}\rho_{\mathrm{los}} d_{u',ES}^{-\varepsilon} + N_0}\right) \tag{8}$$

$$d_{u,ES} = \sqrt{h_u^2 + (x_u - x_{ES})^2 + (y_u - y_{ES})^2} \tag{9}$$

where $d_{u,ES}$ denotes the Euclidean distance between the UAV u and MEC server ES, the path loss between UAV and MEC is $\rho_{\mathrm{los}} d_{u,ES}^{-\varepsilon}$. x_{ES} and y_{ES} are the coordinates of MEC server ES. p_u denotes the transmission power of UAV u. $\sum_{u'=1,u'\neq u}^{U} p_{u'}\rho_{\mathrm{los}} d_{u',ES}^{-\varepsilon}$ is the interference of other UAVs on UAV u.

Furthermore, the downlink data transmission rate, i.e., the data transmission rate from MEC server ES to UAV u is

$$r_{u,ES}^{\mathrm{down}} = B\log_2\left(1 + \frac{p_{ES}\rho_{\mathrm{los}} d_{u,ES}^{-\varepsilon}}{N_0}\right) \tag{10}$$

where p_{ES} denotes the transmission power of MEC server ES,

C. Computation Resources Allocation Model

For each computation node, its computation resource will be allocated based on the size of each subtask. Specifically, the computation resource allocation coefficient of vehicle v for subtask $y_{v,k}$ is the form of

$$\eta_{v,k,v'} = \frac{\beta_{v,k,v'} s_{v,k}}{\sum_{v''\in\mathcal{V}}\sum_{k=1}^{K_{v''}} \beta_{v'',k,v'} s_{v'',k}} \tag{11}$$

where $\beta_{v,k,v'}$ is the offloading of subtask $y_{v,k}$ on vehicle v'. $\beta_{v,k,v'} = 1$, if the subtask $y_{v,k}$ is offloaded to vehicle v'; otherwise, $\beta_{v,k,v'} = 0$ [43].

So, the computation resources of vehicle v' allocated to subtask $y_{v,k}$ is

$$f_{v,k,v'} = \eta_{v,k,v'} f_{v'} \tag{12}$$

where $f_{v'}$ represents the computation resources of vehicle v'.

Similarly, when the subtask $y_{v,k}$ is offloaded to UAV u or MEC server ES, the computation resource allocation coefficient of UAVs and MEC server for the subtask $y_{v,k}$ is denoted as

$$\eta_{v,k,u} = \frac{\beta_{v,k,u} s_{v,k}}{\sum_{v'\in\mathcal{V}}\sum_{k=1}^{K_{i'}} \beta_{v',k,u} s_{v',k}} \tag{13}$$

and

$$\eta_{v,k,ES} = \frac{\beta_{v,k,u}\alpha_{v,k,u,ES} s_{v,k}}{\sum_{v'\in\mathcal{V}}\sum_{k=1}^{K_{i'}} \beta_{v',k,u}\alpha_{v',k,u,ES} s_{v,k}} \tag{14}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

respectively, where $\beta_{v,k,u}$ is the decision variable of vehicle that the subtask $y_{v,k}$ is offloaded to UAV u. $\beta_{v,k,u} = 1$, if the subtask $y_{v,k}$ is offloaded to UAV u from vehicle v; otherwise, $\beta_{v,k,u} = 0$. $\alpha_{v,k,u,ES}$ is the decision variable of UAV that the subtask $y_{v,k}$ from UAV u is offloaded to MEC server ES. $\alpha_{v,k,u,ES} = 1$, if the subtask $y_{v,k}$ is offloaded to MEC server ES from UAV u; otherwise, $\alpha_{v,k,u,ES} = 0$.

The computation resources of UAV u allocated to subtask $y_{v,k}$ are

$$f_{v,k,u} = \eta_{v,k,u} f_u. \tag{15}$$

The computation resources of MEC server ES allocated to subtask $y_{v,k}$ are

$$f_{v,k,ES} = \eta_{v,k,ES} f_{ES} \tag{16}$$

where f_u and f_{ES} represent the computation resources of UAV u and MEC server ES, respectively.

D. Computation Model

In the UAV-assisted IoV environment, a subtask can be either executed locally or offloaded to the other vehicle, UAVs, MEC server. There are three types of computation model, i.e., the local computing model, the vehicle computing model and the UAV computing model. Specifically, these models are introduced as follows.

1) Local Computing Model: The computing delay and energy consumption to process the subtask $y_{v,k}$ on the vehicle v are

$$t_{v,k}^{\text{exe}} = \frac{w_{v,k}}{f_{v,k,v}} \tag{17}$$

$$e_{v,k}^{\text{local,exe}} = \frac{p_v w_{v,k}}{f_{v,k,v}} \tag{18}$$

successively.

Due to the serial correlation between subtasks, one subtask must be processed after the previous subtasks are finished. So, the waiting delay of subtask $y_{v,k}$ is

$$t_{v,k}^{\text{wait}} = t_{v,k}^{\text{pre,total}} \tag{19}$$

where $t_{v,k}^{\text{pre,total}}$ is the previous subtasks delay of subtask $y_{v,k}$.

So, the process time of subtask $y_{v,k}$ is

$$t_{v,k}^{\text{local}} = t_{v,k}^{\text{wait}} + t_{v,k}^{\text{exe}}. \tag{20}$$

When $y_{v,k}$ is executed locally and the content required by subtask $y_{v,k}$ is cached locally, the transmission delay is zero. If the content required by subtask $y_{v,k}$ is cached on other vehicles or UAVs, the vehicles or UAVs with the minimum transmission delay is selected to transmit the corresponding content. If the content required by subtask $y_{v,k}$ is not cached on any vehicles or UAVs but on MEC server, the content required by subtask $y_{v,k}$ is transmitted to the corresponding node from MEC. If subtask $y_{v,k}$ is not cached on any vehicle, UAV, or MEC server, the content required by subtask $y_{v,k}$ is transmitted to the corresponding node from the cloud.

If the content required by subtask $y_{v,k}$ is cached on other vehicles or UAVs, the corresponding transmission delay is

$$z_{v,k,u} = \min_{v' \in \mathcal{V}, \text{hit}_{v,k,v'}=1, \text{hit}_{v,k,u}=1} \left[\text{hit}_{v,k,v'} \left(\frac{o_{v,k}}{r_{v',v}} \right) \text{hit}_{v,k,u} \left(\frac{o_{v,k}}{r_{v,u}^{\text{down}}} \right) \right]. \tag{21}$$

The energy consumption for transmitting the required content from vehicle v to v'' is

$$e_{v',k,v''}^{v2v,c} = p_v \frac{o_{v',k}}{r_{v,v''}} \ \forall v \neq v''. \tag{22}$$

The energy consumption for transmitting the required content from UAV u to vehicle v' is

$$e_{v,k,v'}^{v2u,c} = \frac{p_u o_{v,k}}{r_{v',u}^{\text{down}}}. \tag{23}$$

When the content required by subtask $y_{v,k}$ is cached on other vehicles or UAVs, the total task process delay of subtask $y_{v,k}$ is

$$t_{v,k,u}^{\text{local},vv,vu} = t_{v,k}^{\text{local}} + z_{v,k,u}. \tag{24}$$

When the content required by subtask $y_{v,k}$ is cached on MEC server, the total task process delay is

$$t_{v,k}^{\text{local},m} = t_{v,k}^{\text{local}} + \frac{o_{v,k}}{r_{v,u}^{\text{down}}} + \frac{o_{v,k}}{r_{u,ES}^{\text{down}}}. \tag{25}$$

When the content required by subtask $y_{v,k}$ is not cached on any vehicles, UAVs, or MEC server, the total task process delay of subtask $y_{v,k}$ is

$$t_{v,k}^{\text{local},c} = t_{v,k}^{\text{local}} + \frac{o_{v,k}}{r_{v,u}^{\text{down}}} + \frac{o_{v,k}}{r_{u,ES}^{\text{down}}} + \frac{o_{v,k}}{\nu_{\text{fiber}}} \tag{26}$$

where ν_{fiber} is the data transmission rate from cloud to MEC server.

Therefore, the total local computation delay of subtask $y_{v,k}$ can be represented as

$$t_{v,k}^{l} = \text{hit}_{v,k,v} t_{v,k}^{\text{local}} + a_{v,k,ES} t_{v,k}^{\text{local},m} + b_{v,k} t_{v,k}^{\text{local},c} + \text{hit}_{v,k,\text{arcf}_{v,k,u}} (1 - \text{hit}_{v,k,v}) t_{v,k,u}^{\text{local},vv,vu} \tag{27}$$

where $a_{v,k,ES} = \text{hit}_{v,k,ES} \prod_{u \in U} (1 - \text{hit}_{v,k,u}) \prod_{v' \in V} (1 - hit_{v,k,v'})$ represents that the subtask $y_{v,k}$ is cached on the MEC server not on the vehicles or UAVs. $b_{v,k} = (1 - hit_{v,k,ES}) \prod_{u \in U} (1 - hit_{v,k,u}) \prod_{v' \in V} (1 - hit_{v,k,v'})$ represents that the subtask $y_{v,k}$ is not cached on any vehicles, UAVs, or MEC.

2) Vehicle Computing Model: When subtask $y_{v,k}$ is offloaded to another vehicle v', the computing delay and energy consumption are

$$t_{v,k,v'}^{\text{exe}} = \frac{w_{v,k}}{f_{v,k,v'}} \tag{28}$$

$$e_{v',k',v}^{v2v,\text{exe}} = p_v \frac{w_{v',k'}}{f_{v',k',v}} \tag{29}$$

respectively.

Due to the serial correlation between subtasks, the waiting delay of subtask $y_{v,k}$ is determined by the maximum value between the transmission delay and the total delay of the previous subtask. The waiting delay can be represented as

$$t_{v,k,v'}^{\text{wait}} = \max \left[t_{v,k}^{\text{pre,total}}, t_{v,k,v'}^{\text{tra}} \right] \tag{30}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

where

$$t^{\text{tra}}_{v,k,v'} = \frac{s_{v,k}}{r_{v,v'}}. \tag{31}$$

The energy consumption for transmission the subtask $y_{v,k}$ from vehicle v to v' is

$$e^{v2v,tra}_{v,k,v'} = \frac{p_v s_{v,k}}{r_{v,v'}}. \tag{32}$$

So, the process time of subtask $y_{v,k}$ time is

$$t^{v2v}_{v,k,v'} = t^{\text{wait}}_{v,k,v'} + t^{\text{exe}}_{v,k,v'}. \tag{33}$$

Similar to local computation, it is also necessary to check where the content required by subtask $y_{v,k}$ is cached. If content required by subtask $y_{v,k}$ is cached on other vehicles or UAV, the transmission delay of content required by subtask $y_{v,k}$ is

$$z_{v,k,u} = \min_{v' \in \mathcal{V}, \text{hit}_{v,k,v'}=1, \text{hit}_{v,k,u}=1} \left[\text{hit}_{v,k,v'} \left(\frac{o_{v,k}}{r_{v',v}} \right), \text{hit}_{v,k,u} \left(\frac{o_{v,k}}{r^{\text{down}}_{v,u}} \right) \right]. \tag{34}$$

When the content required by subtask $y_{v,k}$ is cached on other vehicles or UAVs, the total task process delay is

$$t^{v2v,vv,vu}_{v,k,v',u} = t^{v2v}_{v,k.v'} + g_{v,k,u}. \tag{35}$$

When the content required by subtask $y_{v,k}$ is cached on MEC server, the total task process delay is

$$t^{v2v,m}_{v,k,v'} = t^{v2v}_{v,k.v'} + \frac{o_{v,k}}{r^{\text{down}}_{v',u}} + \frac{o_{v,k}}{r^{\text{down}}_{u,ES}}. \tag{36}$$

When the content required by subtask $y_{v,k}$ is not cached on any vehicle, UAV, or MEC server, the total task process delay is

$$t^{v2v,c}_{v,k,v'} = t^{v2v}_{v,k,v'} + \frac{o_{v,k}}{r^{\text{down}}_{v',u}} + \frac{o_{v,k}}{r^{\text{down}}_{u,ES}} + \frac{o_{v,k}}{v_{\text{fiber}}}. \tag{37}$$

Therefore, the total vehicle computation delay of subtask $y_{v,k}$ can be represented as

$$t^{v}_{v,k,v'} = \text{hit}_{v,k,v'} t^{v2v}_{v,k,v'} + a_{v',k,ES} t^{v2v,m}_{v,k,v'} + b_{v',k} t^{v2v,c}_{v,k,v'} + \text{hit}_{v,k,\text{arcg}_{v,k,u}} (1 - \text{hit}_{v,k,v'}) t^{v2v,vv,vu}_{v,k,v',u}. \tag{38}$$

3) UAV Computing Model: $\alpha_{v,k,u,ES}$ is the decision variable that the subtask $y_{v,k}$ is offloaded to MEC server ES. $\alpha_{v,k,u,ES} = 1$, if the subtask $y_{v,k}$ is offloaded to MEC server ES; otherwise, $\alpha_{v,k,u,ES} = 0$. Therefore, when $\alpha_{v,k,u,ES} = 0$, the computation delay of subtask $y_{v,k}$ is

$$t^{\text{exe}}_{v,k,u} = \frac{w_{v,k}}{f_{v,k,u}}. \tag{39}$$

When $\alpha_{v,k,u,ES} = 1$, the computation delay of subtask $y_{v,k}$ is

$$t^{\text{exe}}_{v,k,ES} = \frac{w_{v,k}}{f_{v,k,ES}}. \tag{40}$$

The energy consumption for computing the subtask $y_{v,k}$ by UAV u is

$$e^{v2u,\text{exe}}_{v,k,u} = \frac{p_u w_{v,k}}{f_{v,k,u}}. \tag{41}$$

The transmission delay and energy consumption for transmitting the subtask from vehicle $y_{v,k}$ to UAV u are

$$t^{\text{tra}}_{v,k,u} = \frac{s_{v,k}}{r_{v,u}} \tag{42}$$

$$e^{v2u,\text{tra}}_{v,k,u} = \frac{p_v s_{v,k}}{r_{v,u}} \tag{43}$$

successively.

The transmission delay and energy consumption for transmitting the subtask $y_{v,k}$ from UAV u to MEC server ES are

$$t^{\text{tra}}_{v,k,u,ES} = \frac{s_{v,k}}{r_{u,ES}} \tag{44}$$

$$e^{v2u,u2m,\text{tra}}_{v,k,u,ES} = \frac{p_u s_{v,k}}{r_{u,ES}} \tag{45}$$

successively.

When $\alpha_{v,k,u,ES} = 0$, the waiting delay and the process delay of subtask $y_{v,k}$ are

$$t^{\text{wait},v2u}_{v,k,u} = \max_{\text{pre} \in \text{pred}(y_{v,k})} \left[t^{\text{pre,total}}_{v,k}, t^{\text{tra}}_{v,k,u} \right] \tag{46}$$

$$t^{v2u}_{v,k,u} = t^{\text{wait},v2u}_{v,k,u} + t^{\text{exe}}_{v,k,u} \tag{47}$$

respectively.

When $\alpha_{v,k,u,ES} = 1$, the waiting delay and the process delay of subtask $y_{v,k}$ are

$$t^{\text{wait},v2u2m}_{v,k,u,ES} = \max_{\text{pre} \in \text{pred}(y_{v,k})} \left[t^{\text{pre,total}}_{v,k}, t^{\text{tra}}_{v,k,u} + t^{\text{tra}}_{v,k,u,ES} \right] \tag{48}$$

$$t^{v2u2m}_{v,k,u,ES} = t^{\text{wait},v2u2m}_{v,k,u,ES} + t^{\text{exe}}_{v,k,ES} \tag{49}$$

respectively.

If the content required by subtask $y_{v,k}$ is not cached on the UAV u but on MEC server ES, meanwhile, $\alpha_{v,k,u,ES} = 0$, the total task process delay of subtask $y_{v,k}$ is

$$t^{v2u,m}_{v,k,u} = t^{v2u}_{v,k,u} + \frac{o_{v,k}}{r^{\text{down}}_{u,ES}}. \tag{50}$$

If the content required by subtask $y_{v,k}$ is not cached on the UAV u or MEC server ES, meanwhile, $\alpha_{v,k,u,ES} = 0$, the total task process delay of subtask $y_{v,k}$ is

$$t^{v2u,c}_{v,k,u} = t^{v2u}_{v,k,u} + \frac{o_{v,k}}{r^{\text{down}}_{u,ES}} + \frac{o_{v,k}}{v_{\text{fiber}}}. \tag{51}$$

If the content required by subtask $y_{v,k}$ is not cached on the MEC server ES, meanwhile, $\alpha_{v,k,u,ES} = 1$, the total process delay of subtask $y_{v,k}$ is

$$t^{v2u2m,c}_{v,k,u,ES} = t^{v2u2m}_{v,k,u,ES} + \frac{o_{v,k}}{v_{\text{fiber}}}. \tag{52}$$

So, the total process delay for offloading the subtask $y_{v,k}$ to UAV u and MEC server ES can be expressed as

$$t^{v2u,\text{total}}_{v,k,u} = (1 - \alpha_{v,k,u,ES}) \Big(\text{hit}_{v,k,u} t^{v2u}_{v,k,u} + \text{hit}_{v,k,ES}(1 - \text{hit}_{v,k,u}) t^{v2u,m}_{v,k,u} + (1 - \text{hit}_{v,k,ES})(1 - \text{hit}_{v,k,u}) t^{v2u,c}_{v,k,u} \Big) \tag{53}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

$$t_{v,k,u,ES}^{v2u2m,\text{total}} = \alpha_{v,k,u,ES}\Big(\text{hit}_{v,k,ES} t_{v,k,u,ES}^{v2u2m} + (1-\text{hit}_{v,k,ES}) t_{v,k,ES}^{v2u2m,c}\Big) \tag{54}$$

respectively.

The total UAV computation delay of subtask $y_{v,k}$ can be represented as

$$t_{v,k,u}^{u} = t_{v,k,u}^{v2u,\text{total}} + t_{v,k,u,ES}^{v2u2m,\text{total}}. \tag{55}$$

Then, the total process delay of task A_v can be described as

$$T_V^{\text{total}} = \sum_{k=1}^{K_v}\sum_{u=1}^{U} \beta_{v,k,v} t_{v,k}^{l} + \beta_{v,k,v'} t_{v,k,v'}^{v} + \beta_{v,k,u} t_{v,k,u}^{u}. \tag{56}$$

The energy consumption for transmitting subtask $y_{v,k}$ and executing subtask $y_{v,k}$ by the UAV is

$$e_{v,k,u}^{v2u} = \big(1-\alpha_{v,k,u,ES}\big) e_{v,k,u}^{v2u,exe} + \alpha_{v,k,u,ES} e_{v,k,u,ES}^{v2u,u2m,tra}. \tag{57}$$

If the UAV transmits the content required by subtask $y_{v,k}$ to vehicles, the UAV will also consume energy. When $arcg_{v,k,u} \in \mathcal{U}$ or $arcf_{v,k,u} \in \mathcal{U}$, the energy consumption for transmitting the content required by subtask $y_{v,k}$ from UAV u to vehicle v is

$$e_{v,k,v'}^{v2u,c} = \frac{p_u o_{v,k}}{r_{v',u}^{\text{down}}}. \tag{58}$$

The total energy consumption of the UAV u is

$$e_u^U = \sum_{v=1}^{V}\sum_{k=1}^{K_v} \beta_{v,k,u} e_{v,k,u}^{v2u} + \sum_{v=1}^{V}\sum_{k=1}^{K_v}\sum_{v'=1}^{V} \beta_{v,k,v'} e_{v,k,v'}^{v2u,c}. \tag{59}$$

IV. Problem Formulation

Within the available energy of UAVs, the total task process delay is minimized by joint optimizing task offloading and content caching. The joint optimization problem of task offloading and content caching for UAV-assisted IoV is described as follows:

$$P1: \min_{\mathbf{hit},\boldsymbol{\beta},\boldsymbol{\alpha}} F(\mathbf{hit}, \boldsymbol{\beta}, \boldsymbol{\alpha}) = \sum_{v=1,v'=1}^{V}\sum_{k=1}^{K_V}\sum_{u=1}^{U} \beta_{v,k,v} t_{v,k}^{l} + \beta_{v,k,v'} t_{v,k,v'}^{v} + \beta_{v,k,u} t_{v,k,u}^{u} \tag{60}$$

s.t.

$$\text{hit}_{v,k,j} \in \{0,1\}\ \forall v \in \mathcal{V}\ \forall k \in \mathcal{K}\ \forall j \in \mathcal{V}\cup\mathcal{U}\cup ES \tag{61}$$

$$\beta_{v,k,j} \in \{0,1\}\ \forall v \in \mathcal{V}\ \forall k \in \mathcal{K}\ \forall j \in \mathcal{V}\cup\mathcal{U}\cup ES \tag{62}$$

$$\sum_{\forall j \in \mathcal{V}\cup\mathcal{U}\cup ES} \beta_{v,k,j} = 1\ \forall v \in \mathcal{V}\ \forall k \in \mathcal{K} \tag{63}$$

$$\alpha_{v,k,u,ES} \in \{0,1\}\ \forall v \in \mathcal{V}\ \forall k \in \mathcal{K}\ \forall u \in \mathcal{U} \tag{64}$$

$$T_v^{\text{total}} \le \tau_v\ \forall v \in \mathcal{V} \tag{65}$$

$$\sum_{v=1}^{V}\sum_{k=1}^{K_v} \text{hit}_{v,k,j} o_{v,k} \le H_j^{\max}\ \forall j \in \mathcal{V}\cup\mathcal{U}\cup ES \tag{66}$$

$$e_u \le E_u^{\max}\ \forall u \in \mathcal{U} \tag{67}$$

where hit denotes the vector of cache decision variables, and β and α are the vector of offloading decision variables of vehicles and UAVs, respectively. Constraint (65) ensures that the delay of tasks process does not exceed the maximum available delay. Constraint (66) ensures that the cache content does not exceed the maximum available storage capacity of the each node. Constraint (67) implies that the energy consumption of UAV cannot exceed it available energy. Constraints (59)–(62) limit its range by setting upper and lower bounds, while (63)–(65) limit its range by setting penalty values.

Since the decision variables are all discrete and (60) and (67) are nonlinear, $P1$ is a nonlinear integer programming (NIP) problem. It is quite challenging to optimally address this problem due to the high coupling and high time complexity caused by (60) and (67). Meanwhile, the problem $P1$ is an NP-hard problem [44].

The main methods for solving optimization problems include heuristics, game theory, reinforcement learning, etc. Heuristic algorithms can make use of experience effectively in solving problems, so as to find the approximate optimal solution of complex problems in limited time. However, in the face of complex optimization problems, the algorithm may encounter slow convergence and fall into local optimal solutions [45]. Game theory is used to analyze interactions between multiple players. Because the game theory approach is able to find the optimal strategy in the current situation, it is better suited to dealing with the dynamic behavior of mobile devices, leaving participants with no incentive to deviate unilaterally. But game theory can only ensure that a Nash equilibrium is reached, which is not a globally optimal solution [46]. Reinforcement learning can continuously optimize behavioral strategies by learning past data to obtain the optimal strategy. In addition, reinforcement learning can be flexibly combined with heuristic algorithms and game theory to create more efficient decision structures [47].

The reinforcement learning algorithms are effective for solving the NIP problem due to their adaptability in dynamic and complex environments. Commonly used reinforcement learning algorithm includes Q-learning, DQN, DDPG, and so on. The Q-learning algorithm guides the agent to choose the optimal action by learning the action-value function, which is suitable for the problem of small state–action space. However, in the face of a high-dimensional state space, Q-learning needs to traverse all state–actions, which increases the cost of computation and storage. The DQN algorithm uses a Q-value table or a neural network to represent the action-value function, but the Q-value table is unable to store and retrieve values for all possible state–action pairs in high dimensional problems [48]. Q-learning and DQN can be used to solve discrete problems. But during the training process, the action space of Q-learning and DQN is too complicated, resulting in the slow convergence of Q-learning and DQN training. In contrast to Q-learning and DQN, DDPG is designed for discrete or continuous problems. DDPG mainly uses a network of actors and critics. The actor network is used to output the optimal action by updating the policy gradient and selecting the action that yields the maximum reward. The critic network is for estimating the value function, which evaluates the

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

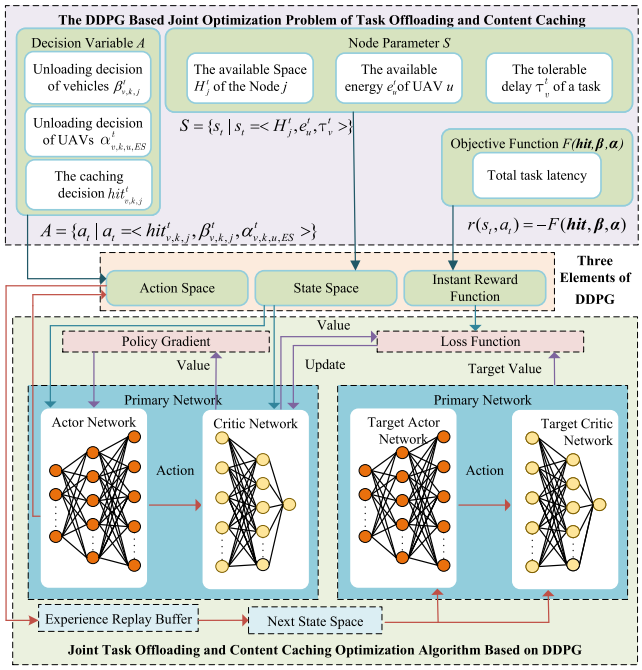


Fig. 4. Relationship between the proposed optimization problem and DDPG.

expected return for each action [49]. The DDPG algorithm can effectively handle the challenges of large action spaces. Since our decision variables are discrete and the optimization problems are NIP problems. Meanwhile, the action spaces of our optimization problem are large. Therefore, DDPG can solve our optimization problems better than Q-learning and DQN.

V. Proposed Algorithm

The DDPG-JOTOCC algorithm uses the reinforcement learning Actor–Critic architecture, which contains three modules: 1) target network; 2) main network; and 3) replay buffer. The main network generates specific task offloading and migration decisions based on the current state s_t. The main network consists of two deep neural networks (DNNs): 1) the Actor_M network $\mu(s_t|\theta^{\mu})$ and 2) the Critic_M network $Q(s_t, a_t|\theta^{Q})$. The target network generates a target value for training the Critic_M network, which has the same structure as the main network but different parameters. Actor_T network and Critic_T network are represented by $\mu'(s_t|\theta^{\mu'})$ and $Q'(s_t, a_t|\theta^{Q'})$. The relationship between the proposed optimization problem and DDPG is shown in Fig. 4.

A. Markov Decision

The objective $P1$ operates within a single time slot environment, each action taken generates different available space of the node and the cache state is therefore dynamic, so past cache states are also important reference factors. So that, we first formulated the problem as a Markov decision process (MDP), which can be represented by a tuple $\{S, A, P, R\}$, S is the state space of the model, and A is the action space of the model. P is the state transition matrix of the model, and R is the reward function of the model [50].

The state space is defined as

$$S = \{s_t | s_t =< H_j^t, e_u^t, \tau_v^t >\} \tag{68}$$

where H_j^t is the available space of the node j, and $H_j^t = H_j$. e_u is the available energy of UAV u, and $e_u^t = e_u$. τ_v^t is the tolerable delay of a task, and $\tau_v^t = \tau_v$, $t \in \mathcal{T} = \{1, 2, \ldots, T\}$. $\mathcal{T}$ is the set of time slots.

The action space is defined as

$$A = \{a_t | a_t =< \text{hit}_{v,k,j}^t, \beta_{v,k,j}^t, \alpha_{v,k,u,ES}^t >\} \tag{69}$$

where $hit_{v,k,j}^t$ is the decision variable of content required by the subtask $y_{v,k}$ on the computation node j, and $\text{hit}_{v,k,j}^t = hit_{v,k,j}$. $\beta_{v,k,j}^t$ is the decision variable of vehicle that the subtask $y_{v,k}$ is offloaded to UAV u, and $\beta_{v,k,j}^t = \beta_{v,k,j}$. $\alpha_{v,k,u,ES}^t$ is the decision variable of UAV that the subtask $y_{v,k}$ from UAV u is offloaded to MEC server ES, and $\alpha_{v,k,u,ES}^t = \alpha_{v,k,u,ES}$.

The definition of state transition probability is as follows. The state transition of the environment and the rewards obtained have the Markov feature, that is, the probability and rewards of state transition depend on the state space S and the action A. The state transition is $P(s_t, a_{s',t}, s')$ stored in the replay memory for training the neural networks, it defines the probabilities of transition state $s' \in S$ from state $s_t \in S$ after taking action $a_{s',t} \in A$.

$$P(s_t, a_{s',t}, s') = p \tag{70}$$

$$P(s_t, a_{s',t}, s_{t'}) = 1 - p. \tag{71}$$

Equation (70) indicates the probability of transition from state s_t to state s'. Equation (71) indicates the MDP goes to the termination state $s_{t'} \in S$ when the transition to state s' fails from the initialization state s_0 [51]. This represents the uncertainty in offloading.

The definition of the reward function is as follows. The goal of the optimization $P1$ is to minimize the total task process delay. So, the reward function is denoted by the negative total task process delay, i.e.,

$$r(s_t, a_t) = -F(\mathbf{hit}, \boldsymbol{\beta}, \boldsymbol{\alpha}). \tag{72}$$

So the total reward is $R = \sum_{t=1}^{T} r(s_t, a_t, s_{t+1})$. Our ultimate goal is to maximin the total reward.

B. DDPG-JOTOCC Algorithm

The pseudocode of the DDPG-JOTOCC algorithm is shown in Algorithm 1. At the beginning, communication between vehicles and UAVs is established (Algorithm 1, lines 1–7). The system state s_0 is the first input of actor network. The actor network generates a resource allocation action a_t for the current system state s_t (Algorithm 1, lines 13–15). Once action a_t is obtained, the resource allocation and the delay of task can be calculated and processed (Algorithm 1, lines 16–18). Based on the action a_t, the reward can be calculated by (63), the resulting system state s_{t+1} can be observed and the experience (s_t, a_t, r_t, s_{t+1}) is stored into the experience replay memory D_n (Algorithm 1, lines 20 and 21).

In the target Actor network $\mu'(s_t|\theta^{\mu'})$ and the target Critic network $Q'(s_t, a_t|\theta^{Q'})$, the parameters will be updated by

$$\theta^{\mu'} \leftarrow \psi\theta^{\mu} + (1 - \psi)\theta^{\mu'} \tag{73}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 DDPG-JOTOCC Algorithm

Input: The initial value of Critic_M network $Q(s_t, a_t|\theta^Q)$ with random parameter θ^Q, the initial value of Actor_M network $\mu(s_t|\theta^\mu)$ with random parameter θ^μ, the initial value of Critic_T network $Q'(s_t, a_t|\theta^{Q'})$ with random parameter $\theta^{Q'}$, the initial value of Actor_T network $\mu'(s_t|\theta^{\mu'})$ with random parameter $\theta^{\mu'}$.
Output: Offloading decision $\boldsymbol{\beta}$ and $\boldsymbol{\alpha}$, the caching decision ***hit***.
1: **For** each vehicle $v \in \mathcal{V}$ do
2: Calculate $r_{v,v'}$ // The wireless data transmission rate between vehicle v and vehicle v'
3: Calculate $r_{v,u}$ // The uplink and downlink data transmission rate between vehicle v and UAV u
4: **End For**
5: **For** each UAV $u \in \mathcal{U}$ do
6: Calculate $r_{u,ES}$ // The uplink and downlink data transmission rate between the UAV u and MEC
7: **End For**
8: **For** $n = 1$ to episode N do
9: Obtain initial state s_0.
10: **For** time slot $t = 1, 2, ..., T$ do
11: Obtain action a_t and state s_t by (68) and (69).
12: **For** each subtask $y_{v,k}$ do
13: Calculate $f_{v,k,v'}$ by (12) // The computation resources of vehicle v' allocated to subtask $y_{v,k}$
14: Calculate $f_{v,k,u}$ by (15) // The computation resources of UAV u allocated to subtask $y_{v,k}$
15: Calculate $f_{v,k,ES}$ by (16) // The computation resources of MEC server ES allocated to subtask $y_{v,k}$
16: Calculate $t^l_{v,k}$ by (27) // The total delay of local computation
17: Calculate $t^v_{v,k,v'}$ by (38) // The total delay of V2V computation
18: Calculate $t^u_{v,k,u}$ by (55) // The total delay of V2U computation
19: **End For**
20: Update the reward r_t by (65) and observe the new state s_{t+1}.
21: Store the experience (s_t, a_t, r_t, s_{t+1}) into the replay buffer D_n.
22: Sample randomly a minibatch of D_z experience (s_m, a_m, r_m, s_{m+1}) from D_n.
23: Set $y_m = r_m + \gamma Q'(s_{m+1}, \mu'(s_{m+1}; \theta^{\mu'})|\theta^{Q'})$.
24: Update Critic_M by minimizing the loss, i.e., $Loss = \frac{1}{D_z}\sum_{m=1}^{D_z}\left(y_m - Q(s_m, a_m|\theta^Q)\right)^2$.
25: Update Actor_M policy using the sampled gradient:
26: $\nabla_{\theta^\mu} J \approx \frac{1}{D_z}\sum_{m=1}^{D_z}\left(\nabla_a Q(s_m, a_m|\theta^Q)\Big|_{a_m=\mu(s_m|\theta^\mu)} \nabla_{\theta^\mu}\mu(s_m|\theta^\mu)\right)$
27: Update the Critic_T network and Actor_T network:
28: $\theta^{Q'} \leftarrow \psi\theta^Q + (1-\psi)\theta^{Q'}$
29: $\theta^{\mu'} \leftarrow \psi\theta^\mu + (1-\psi)\theta^{\mu'}$.
30: **End for**
31: **End for**
32: **return** $\boldsymbol{\beta}$, $\boldsymbol{\alpha}$, ***hit***

and

$$\theta^{Q'} \leftarrow \psi\theta^Q + (1-\psi)\theta^{Q'} \tag{74}$$

respectively, where ψ is the soft update factor (Algorithm 1, lines 28 and 29).

The time complexity of the DDPG-JOTOCC algorithm depends on DNNs in the actor and the critic. The DNN dimensions for the actor's input and output are $5 + 2V + U$ and $2K(1 + V + U)$, respectively, while for the critic, the dimensions are $5 + 2V + U$ and 1, where V is the number of vehicles, U is the number of UAVs, and K is the total number of subtasks. The number of hidden layers in both the actor and critic is equal and denoted by L, B is the batch size. Thus, the complexity of gradient descent of the actor and the critic is estimated as $O(BLK(V+U)^2)$ and $O(BL(V+U))$, respectively. So, the complexity of the DDPG-JOTOCC algorithm is estimated as $O(BLK(V+U)^2)$, where E is the number of episodes.

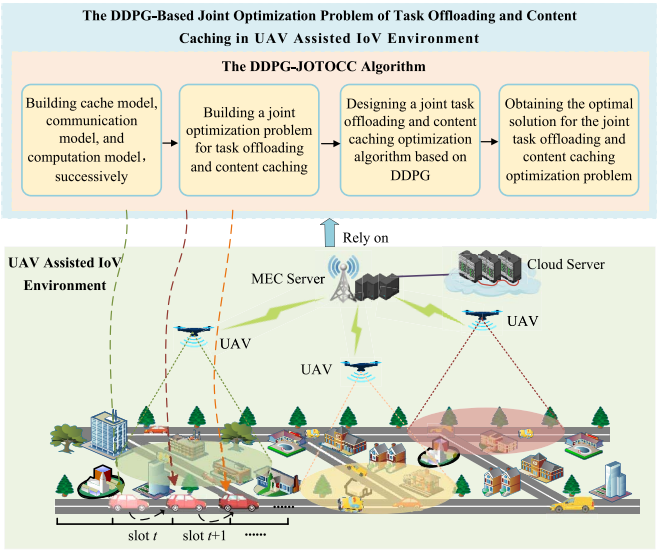


Fig. 5. Application scenario for the DDPG-JOTOCC algorithm.

VI. Use Case

At present, Tesla's autopilot and full auto drive system have achieved L2 and L3 level autopilot, which can provide auxiliary functions, such as automatic braking and automatic driving. However, the camera, laser radar, and other sensors in the auto drive system will produce a large number of high-resolution data in the environmental information acquisition phase, and processing these data requires a large amount of storage space and computing resources. In order to optimize resource utilization, The UAVs can cache high-precision map segments and real-time traffic information frequently accessed by vehicles in advance. In this way, vehicles can immediately obtain this data from UAVs when they need it, without having to request it from remote servers every time. By caching this data in the UAVs, latency can be significantly reduced and system response speed can be improved. At the same time, these tasks can also be offloaded to UAVs, which can accelerate processing speed and reduce the burden on onboard systems.

As shown in Fig. 5, a DDPG-JOTOCC algorithm is proposed for UAV-assisted IoV environment to address the problem of high vehicle task processing latency caused by limited computing resources and return links at each node. In the autonomous driving scenario, the quasi-static assumption can be adopted for the time slot within second or millisecond units. During the time slot, the DDPG-JOTOCC algorithm is adopted to cache the high-precision maps or traffic information into vehicles, UAVs or edge servers and allocate the corresponding resources for reducing response delay of vehicle service. During different time slot, the change of application scenario state will incur the change of input

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

TABLE III
SIMULATION PARAMETER SETTING

Description	Parameters	Value
The cache content size for subtask $y_{v,k}$	$o_{v,k}$	[0.5,1] Mb
The size of task A_v	d_v	[100,300]KB [52]
The sending power of MEC server	p_{ES}	40W [53]
The cache capacities of MEC server	H_{ES}	100MB
The sending power of UAV u	p_u	0.2W
The cache capacities of UAV u	H_u	80MB
The computation resources of UAV u	f_{uav}	[6, 8]GHz
The number of CPU clocks cycles required to execute $y_{v,k}$	$w_{v,k}$	[10,1000]cycles/bit [54]
The shadow fading component	ρ_{los}	-80dBm
The path loss exponent	ε	2
The computation resources of vehicle v	f_v	[1, 2] GHz
The downlink bandwidth of MEC server	B_{ES}^{down}	1000MHz
The downlink bandwidth of UAV u	B_u^{down}	10MHz
The noise power	N_0	-114dBm
The channel bandwidth	B	30MHz
The sending power of vehicle v	p_v	1.3W
The cache capacities of vehicle v	H_v	60MB
The learning rate for actor	ω_a	0.001
The learning rate for critic	ω_c	0.002
The reward discount	γ	0.9
The soft replacement	ς	0.01
The memory capacity	ψ	5000

parameters of DDPG-JOTOCC algorithm. Then, the DDPG-JOTOCC algorithm is again used to cache the contents and allocate the corresponding resources required by vehicles. By this method, the available resources are reasonably allocated by using the DDPG-JOTOCC algorithm for each time slot to improve the vehicle service quality.

VII. NUMERICAL SIMULATION

In this section, the extensive experiments are conducted to verify the performance of the DDPG-JOTOCC algorithm. First, the experimental environment is set. Then, the benchmark algorithms are described. Finally, the numerical results are analyzed. Specifically, we conducted the simulation experiment on a single computer. Meanwhile, the simulation results are obtained by conducting 20 repeated experiments to calculate the average value.

A. Experiment Environment

The performance of the DDPG-JOTOCC algorithm is verified on one personal computer, whose configuration information is Intel Core i7-12700f CPU@2.10 GHz, RAM 16.0 GB, DISK 1024 GB, and Window10 operating system. The python language is adopted, whose version is Python3.8, the python-integrated development environment is Pycharm2020, in which Anaconda 4.10.1 is adopted to build the virtual environment. The process of training and prediction is conducted based on Tensorflow 2.3. For the performance verification of the DDPG-JOTOCC algorithm, the parameter setting for simulation is given in Table III, where the cache capacities of MEC server *ES*, UAV *u*, and vehicle *v* are represented as H_{ES}, H_u, and H_v, respectively.

B. Benchmark Algorithm

To verify the performance of DDPG-JOTOCC algorithm, the DDPG-based popularity content caching and task offloading (DDPG-PCCTO) algorithm [55], the DDPG-based variety content caching and task offloading (DDPG-VCCTO) algorithm [56], the DDPG-based task offloading algorithm without vehicles communication and the content caching (DDPG-TOWVCC) [57], and the DDPG-based content caching and task offloading with UAV relay (DDPG-CCTOR) algorithm [58] are taken as the benchmark algorithms, successively. The offloading strategy of DDPG-PCCTO algorithm is the same as DDPG-JOTOCC algorithm, but the caching strategy adopts the popularity content caching. The offloading policy of DDPG-VCCTO algorithm is identical to DDPG-JOTOCC algorithm and the caching strategy employs variety content caching, which cache as much contents as possible. In DDPG-TOWVCC algorithm, the task cannot be offloaded to other vehicles, and the resource provisioning nodes cannot cache the content. In DDPG-CCTOR algorithm, the UAV can only transmit tasks to MEC server, but it does not have the computation unit and cannot execute tasks by itself. Specifically, the differentiation among benchmark algorithms is shown in Table IV.

In addition, total delay for nonviolated tasks (TDVTs), TCR, the total delay of all tasks (TDATs) [59] and TVEO [60] are taken as the metrics. TDAT denotes the sum of the TDATs after completion. TDVT represents the sum of the delay of nonviolated task after completion. TCR represents the ratio of the number of tasks completed within the maximum tolerated time to the total number of tasks. TVEO represents the total energy consumption of vehicles during the process of handling and transmitting tasks.

C. Experimental Results

In this section, the feasibility of DDPG-JOTOCC algorithm is verified. Furthermore, the influence of the number of vehicles, the computation resource of vehicles, the number of UAVs and the size of task on the performance of DDPG-JOTOCC algorithm is discussed, successively. In the verification experiment, the number of vehicles, the computation resource of vehicles, the number of UAVs, and the size of task are set to 20, [1, 2] GHz, 7, and [100, 300] kB, successively.

To verify the feasibility of DDPG-JOTOCC algorithm, the algorithm convergence is achieved by simulation experiments, and the average value of 20 simulation results is taken to plot Fig. 6. Fig. 6 describes the convergence of DDPG-JOTOCC algorithm. With the increase of iteration times, all decision variables are gradually optimized and adjusted, and the TDAT tends to be convergent.

To discuss the influence of the number of vehicles on the performance of DDPG-JOTOCC algorithm, the number of vehicles is set to 10, 15, 20, 25, 30, and 35, successively. The number of UAVs is 7, and the size of task is set to [100, 300] kB, the computation resource of vehicles is [1, 2] GHz. Then, the experimental results are achieved as shown in Fig. 7.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
DIFFERENTIATION AMONG BENCHMARK ALGORITHMS

Ref.	Algorithm	Scenario	Considering Factor		Output
			Caching strategy	Offloading policy	
[55]	The DDPG-based popularity content caching and task offloading algorithm	MEC assisted IoT	Popular contents	Same as that in our proposed algorithm	The offloading decisions for all nodes, such as vehicles, UAVs and edge server
[56]	The DDPG- based variety content caching and task offloading algorithm	MEC	Variety content caching	Same as that in our proposed algorithm	The offloading decisions for all nodes
[57]	The DDPG-based task offloading algorithm without communication between vehicles	UAV assisted MEC	Not caching	Same as that in our proposed algorithm	The caching and offloading decisions for all nodes except for vehicles
[58]	The DDPG-based content caching and task offloading with UAV relay algorithm	D2D assisted MEC	Same as that in our proposed algorithm	Same as that in our proposed algorithm	The caching and offloading decisions for all nodes except for UAVs
	Proposed algorithm	UAV assisted IoV	Dynamic adjustment by proposed algorithm	Dynamic adjustment by proposed algorithm	The caching and offloading decisions for all nodes

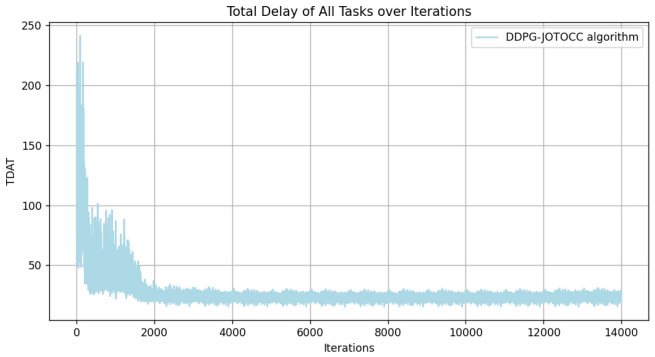


Fig. 6. Convergence of the DDPG-JOTOCC algorithm.

From Fig. 7(a), it is known that TDVT increases with the number of vehicles increasing. In this case, when the number of vehicles becomes large, the number of tasks is increased. Due to the limit cache space and computation resource of vehicles, the delay of nonviolated task process increases. Then, the TDVT increases. When the number of vehicles is 30, the TDVT of DDPG-JOTOCC algorithm is 35.00 s. Comparing with the DDPG-JOTOCC algorithm, the TDVT of DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms is reduced up to 41.46%, 23.69%, 61.97%, and 71.57%, respectively. The DDPG-JOTOCC algorithm jointly optimizes the task offloading and the content caching. So, the delay of task process and transmission decreases. The caching strategy of DDPG-PCCTO algorithm adopts the popularity content caching, and the caching strategy of DDPG-VCCTO algorithm employs variety content caching. So, in the DDPG-PCCTO and DDPG-VCCTO algorithms, the caching strategy is not influenced by the offloading strategy, the decision of offloading and caching is not the optimal solution, which increases the delay of nonviolated task process. In DDPG-TOWVCC algorithm, the task cannot be offloaded to other vehicles, and the node cannot cache the content. In DDPG-CCTOR algorithm, the UAV cannot execute tasks. In DDPG-TOWVCC and DDPG-CCTOR algorithms, due to the decrease of resource provisioning nodes, the computing resources is decreased and the delay of nonviolated task process is increased. So, the performance of DDPG-JOTOCC algorithm on TDVT is better than that of the benchmark algorithms.

Fig. 7(b) shows that the TCR becomes small with the increase of the number of vehicles. With the increase of the number of vehicles, the number of task and the variety of requested content become large. In this case, due to the limit cache space and computation resource of each vehicle, the delay of task process will increase and the number of nonviolated tasks process is decreased. Meanwhile, the TCR is reduced. When the number of vehicles is 30, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, the TCR of DDPG-JOTOCC algorithm is raised up to 16.67%, 10.00%, 30.00%, and 30.00%, respectively. The offloading decision of the DDPG-PCCTO and DDPG-VCCTO algorithms is the same as the DDPG-JOTOCC algorithm, but the caching scheme of DDPG-PCCTO and DDPG-VCCTO algorithms is popularity content caching and variety content caching, respectively. So, the decision is not the optimal solution in the DDPG-PCCTO and DDPG-VCCTO algorithms, and the delay of task process is increased in the DDPG-PCCTO and DDPG-VCCTO algorithms which incurs the decrease of the number of nonviolated tasks process. The caching resource of the DDPG-TOWVCC algorithm is less than that of DDPG-JOTOCC algorithm, the computation resource of the DDPG-CCTOR algorithm is less than that of DDPG-JOTOCC algorithm, the delay of task process of the DDPG-TOWVCC and DDPG-CCTOR algorithm is larger than that of DDPG-JOTOCC algorithm, the number of nonviolated tasks process is decreased. So, the performance of DDPG-JOTOCC algorithm on TCR is better than that of the benchmark algorithms.

Fig. 7(c) implies that the value of TDAT becomes large with the increase of the number of vehicles. When the number of tasks is increasing, due to the limited computation and caching resources, the delay of task process is increased, so the TDAT becomes large. When the number of vehicles is 20, the TDAT of DDPG-JOTOCC algorithm is higher than that of DDPG-TOWVCC algorithm, this is because the DDPG-JOTOCC

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

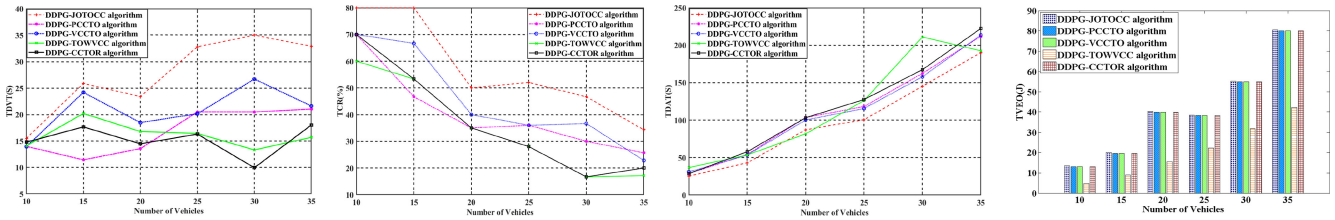


Fig. 7. Influence of the number of vehicles on the algorithm performance. (a) Influence of the number of vehicles on TDVT. (b) Influence of the number of vehicles on TCR. (c) Influence of the number of vehicles on TDAT. (d) Influence of the number of vehicles on TVEO.

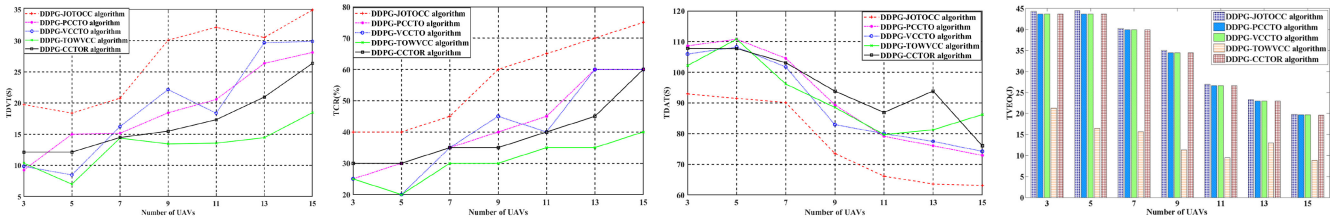


Fig. 8. Influence of the number of UAVs on the algorithm performance. (a) Influence of the number of UAVs on TDVT. (b) Influence of the number of UAVs on TCR. (c) Influence of the number of UAVs on TDAT. (d) Influence of the number of UAVs on TVEO.

algorithm prioritizes ensuring the number of completed tasks, as shown in Fig. 7(a) and (b). In Fig. 7(a), DDPG-JOTOCC algorithm has the highest TDVT than DDPG-TOWVCC algorithm; In Fig. 7(b), DDPG-JOTOCC algorithm has the highest TCR than DDPG-TOWVCC algorithm. Therefore, although the TDAT of DDPG-JOTOCC algorithm is higher than that of DDPG-TOWVCC algorithm, the TDVT and the TCR are the highest. When the number of vehicles is 30, the TDAT of DDPG-JOTOCC algorithm is 145.2 s. Comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, DDPG-JOTOCC algorithm can achieve 10.54%, 7.98%, 31.18% and 13.21% reduction of TDAT, successively. The caching strategy of the DDPG-PCCTO and DDPG-VCCTO algorithm is popularity content caching and variety content caching, respectively, the caching strategy is not the optimal solution. So, the transmission delay of task of DDPG-PCCTO and DDPG-VCCTO algorithm is larger than that of DDPG-JOTOCC algorithm. In the DDPG-TOWVCC algorithm, the caching resource is less than that of DDPG-JOTOCC algorithm, so that the transmission delay of the task of the DDPG-TOWVCC algorithm is larger than that of DDPG-JOTOCC algorithm. In the DDPG-CCTOR algorithm, the computation resource is less than that of DDPG-JOTOCC algorithm, so that the delay of the task process of the DDPG-CCTOR algorithm is larger than that of DDPG-JOTOCC algorithm. So, the performance of the DDPG-JOTOCC algorithm on TDAT is better than that of the benchmark algorithms.

From Fig. 7(d), it is known that TVEO is increasing with the number of vehicles increasing. When the number of vehicles and the number of tasks become large, the computation resource allocated to the tasks by the vehicles becomes large which incurs the increase of energy consumption, so the TVEO is increased. When the number of vehicles is 30, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, the TVEO of DDPG-JOTOCC algorithm is raised up to 0.76%, 0.75%, 74.27%, and 0.76%, respectively. Because the caching strategy of popularity content caching and variety content caching is not the optimal solution, the energy consumption for transmitting the required contents of subtasks of vehicles becomes small. The computation resource in the DDPG-TOWVCC and DDPG-CCTOR algorithm is less than that of DDPG-JOTOCC algorithm, and the computation resource allocated to the tasks by the vehicles becomes small. Meanwhile, the power of the vehicle is not changed and the energy consumption of vehicles is decreased. So, the performance of the DDPG-JOTOCC algorithm on TVEO is better than that of the benchmark algorithms.

To analyze the influence of the number of UAVs on the algorithm performance, the number of UAVs is set to 3, 5, 7, 9, 11, 13, and 15, successively. The number of vehicles is 20, and the size of task is set to [100.300] kB, the computation resource of vehicles is [1,2] GHz. Then, the experimental results are obtained as shown in Fig. 8.

Fig. 8(a) shows that the TDVT is raising with the number of UAVs increasing. Because the computation resource allocated to tasks by UAVs becomes large with the increase of the number of UAVs, which decreases the delay of nonviolated task process and increases the number of nonviolated tasks. So, TDVT is raised. When the number of UAVs is 9, the TDVT of DDPG-JOTOCC algorithm is 30.07 s, and is raised up to 38.71%, 26.31%, 55.30%, and 48.52%, respectively, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, successively, whose reason is the same as that in Fig. 7(a). From Fig. 8(b), it is known that the TCR is improved with the increase of the number of UAVs. When the number of UAVs becomes large, the computation resource of UAVs is increased and the delay of task process is reduced, which increases the number of nonviolated tasks. So, TCR is raised. When the number of

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

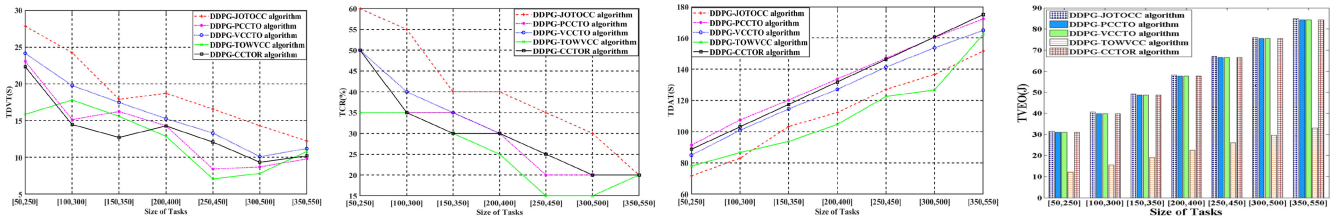


Fig. 9. Influence of the size of tasks on the algorithm performance. (a) Influence of the size of tasks on TDVT. (b) Influence of the size of tasks on TCR. (c) Influence of the size of tasks on TDAT. (d) Influence of the size of tasks on TVEO.

UAVs is 9, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, the TCR of DDPG-JOTOCC algorithm is raised up to 20.00%, 15.00%, 30.00%, and 25.00%, respectively, whose reason is the same as that in Fig. 7(b).

Fig. 8(c) implies that TDAT is reducing with increase of the number of UAVs. The computation resource allocated to the tasks becomes large with the increase of the number of UAVs, which reduces the delay of task process. So, TDAT is reduced. When the number of UAVs is 9, the TDAT of DDPG-JOTOCC algorithm is 73.53 s and is reduced up to 17.71%, 11.33%, 16.94%, and 21.65% than that of DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, respectively, whose reason is the same as that in Fig. 7(c). Fig. 8(d) implies that TVEO is reducing with increase of the number of UAVs. Because the number of tasks assigned to UAVs becomes large with the increase of the number of UAVs, the number of tasks assigned to vehicles becomes small, which reduces the energy consumption of vehicles. So that TVEO is reduced. When the number of UAVs is 9, the TVEO of DDPG-JOTOCC algorithm is 35.03 J and is raised up to 1.75%, 1.75%, 210.02%, and 1.75%, respectively, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, whose reason is the same as that in Fig. 7(d).

To discuss the influence of the size of tasks on the algorithm performance, the size of tasks is set to [50, 250] kB, [100, 300] kB, [150, 350] kB, [200, 400] kB, [250, 450] kB, [300, 500] kB, and [350, 550], successively. Meanwhile, the number of UAVs is set to 7, the number of vehicles is set to 20 and the computation resource of vehicles is [1,2] GHz. The experimental results are obtained as shown in Fig. 9.

Fig. 9(a) shows that the TDVT is reducing with the size of task increasing. When the number of vehicles, UAVs, and computation resource are given, the delay of nonviolated task process becomes large with the increase of size of tasks, which increase the TDVT. When the size of tasks is [100, 300] kB, the TDVT of DDPG-JOTOCC algorithm is 24.23 s. Comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, the TDVT of DDPG-JOTOCC algorithm is raised up to 60.03%, 22.56%, 36.12%, and 67.45%, respectively, whose reason is the same as that in Fig. 7(a). From Fig. 9(b), it is known that the TCR is reduced with the increase of the size of tasks. With the increase of size of tasks, the delay of task process is increased and the total computation resource is not changed, which incurs the decrease of the number of nonviolated tasks. So that TCR is reduced. When the size of tasks is [100, 300] kB, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, the TCR of DDPG-JOTOCC algorithm is raised up to 20.00%, 15.00%, 20.00%, and 30.00%, respectively, whose reason is the same as that in Fig. 7(b).

From Fig. 9(c), it is known that the TDAT becomes small with the increase of the size of tasks. When the computation resource is given, the delay of each task process becomes large with the increase of the size of tasks. So, the TDAT is increased. Sometimes, the TDAT of DDPG-JOTOCC algorithm is larger than that of DDPG-TOWVCC algorithm, for the same reason as that in Fig. 7(c). When the size of task is [100, 300] kB, the TDAT of DDPG-JOTOCC algorithm is reduced up to 22.78%, 17.68%, 4.12%, and 19.48% than that of DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, respectively, whose reason is the same as that in Fig. 5(c). Fig. 9(d) shows that the TVEO is raising with the size of tasks increasing. When the size of tasks becomes large, the total delay of tasks process is increased. Meanwhile, the total computation resource and the power of vehicles are not changed, which increases the energy consumption of vehicle. So, TVEO is raised. When the size of tasks is [100, 300] kB, the TVEO of DDPG-JOTOCC algorithm is 30.07 s and is raised up to 1.76%, 1.76%, 16.94%, and 159.96%, respectively, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, successively, whose reason is the same as that in Fig. 7(d).

To analyze the influence of the computation resource of vehicles on the algorithm performance, the computation resource of vehicles is set to [0.5, 1.5] GHz, [1, 2] GHz, [1.5, 2.5] GHz, [2, 3] GHz, [2.5, 3.5] GHz, and [3, 4] GHz, successively. The number of vehicles is 20, and the size of task is set to [100.300] kB, the number of UAVs is 7. Then, the experimental results are obtained as shown in Fig. 10.

Fig. 10(a) shows that the TDVT is increasing with the computation resource of vehicles increasing. When the number of vehicles, the size of tasks and UAVs are given, the delay of task process become small with the computation resource of vehicles increasing, which increases the total delay of the nonviolated tasks process. So, the TDVT is increased. When the computation resource is [3, 4] GHz, the TDVT of DDPG-JOTOCC algorithm is 38.83 s and is raised up to 70.38%, 28.11%, 142.23%, and 67.80%,

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

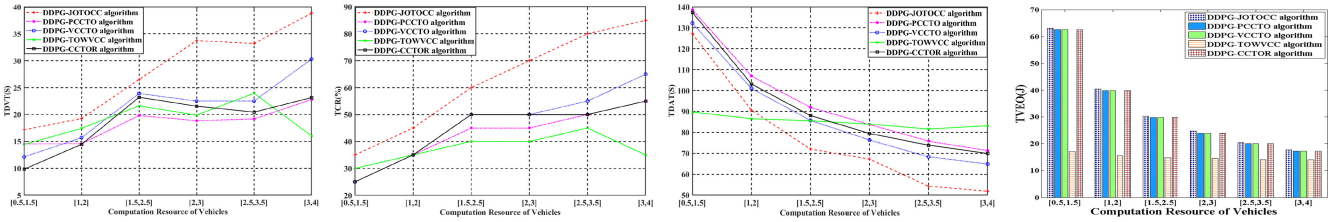


Fig. 10. Influence of the computation resource of vehicles on the algorithm performance. (a) Influence of the computation resource of vehicles on TDVT. (b) Influence of the computation resource of vehicles on TCR. (c) Influence of the computation resource of vehicles on TDAT. (d) Influence of the computation resource of vehicles on TVEO.

respectively, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, whose reason is the same as that in Fig. 7(a). From Fig. 10(b), it is known that the TCR is improved with the increase of the computation resource of vehicles. With the increase of computation resource of vehicles, each task is assigned more computing resources. So, the delay of task process is decreased which incurs the increase of number of nonviolated tasks. So that TCR is raised. When the computation resource is [3, 4] GHz, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, the TCR of DDPG-JOTOCC algorithm is raised up to 30.00%, 20.00%, 50.00%, and 30.00%, respectively, whose reason is the same as that in Fig. 7(b).

Fig. 10(c) implies that TDAT is reducing with increase of computation resource of vehicles. The computation resource allocated to the tasks becomes large with the increase of computation resource, which incurs the decrease of delay of task process. So that TDAT is reduced. Sometimes, the TDAT of DDPG-JOTOCC algorithm is larger than that of DDPG-TOWVCC algorithm, for the same reason as that in Fig. 7(c). When the computation resource is [3, 4] GHz, the TDAT of DDPG-JOTOCC algorithm is 51.80 s, and is reduced up to 27.31%, 20.16%, 37.70%, and 25.90%, respectively, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, whose reason is the same as that in Fig. 7(c). Fig. 10(d) shows that the TVEO is reducing with the computation resource of vehicles increasing. Because the computation resource allocated to tasks becomes large with the increase of the computation resource of vehicles, the delay of tasks process is decreased. Meanwhile, the power of each vehicle is not changed. So, the TVEO is decreased. When the computation resource is [3, 4] GHz, the TVEO of the DDPG-JOTOCC algorithm is 30.07 s and is raised up to 2.78%, 2.78%, 26.84%, and 2.78%, respectively, comparing with DDPG-PCCTO, DDPG-VCCTO, DDPG-TOWVCC, and DDPG-CCTOR algorithms, successively, whose reason is the same as that in Fig. 7(d).

VIII. Conclusion

In this article, the joint optimization strategy of computation offloading and content caching in UAV-assisted IoV environment is proposed to reduce the delay of the task process. The joint optimization problem of computation offloading and content caching is built to minimize the total task process delay based on the caching model, the communication model, the computation resource allocation model, and the computation model in UAV-assisted IoV environment, and for enhancing the performance of task execution and caching strategies in UAV-assisted IoV networking, a DAG model is proposed to formulate the task process. Then, the DDPG-based algorithm is adopted to solve the proposed joint optimization problem, in which the decision variables are taken as the action space and the available resources are regarded as the state space. Meanwhile, the task process delay is taken as the reward function. Moreover, the DDPG-based joint optimization algorithm of task offloading and content caching is designed. In future work, the task of application will be split into multiple parallel and serial subtasks, which increases the applicability of the model. The DDPG-JOTOCC algorithm will be applicated in pervasive environments and the performance of DDPG-JOTOCC algorithm will be evaluated in the actual application environment.

References

[1] Y.-J. Seo, J. Lee, J. Hwang, D. Niyato, H.-S. Park, and J. K. Choi, "A novel joint mobile cache and power management scheme for energy-efficient mobile augmented reality service in mobile edge computing," *IEEE Wireless Commun. Lett.*, vol. 10, no. 5, pp. 1061–1065, May 2021.

[2] X. Hou et al., "Reliable computation offloading for edge-computing-enabled software-defined IoV," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7097–7111, Aug. 2020.

[3] Y. Zhang, M. Zhang, C. Fan, F. Li, and B. Li, "Computing resource allocation scheme of IOV using deep reinforcement learning in edge computing environment," *EURASIP J. Adv. Signal Process.*, vol. 2021, no. 1, p. 33, 2021.

[4] J. Bai, S. Zhu, Y. Chen, and Y. Chen, "Joint optimization of caching and content delivery in air–ground cooperation environment," *IEEE Internet of Things J.*, vol. 12, no. 5, pp. 6029–6045, Mar. 2025

[5] C. Park and J. Lee, "Mobile edge computing-enabled heterogeneous networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 1038–1051, Feb. 2021.

[6] Z. Liu, Y. Cao, P. Gao, X. Hua, D. Zhang, and T. Jiang, "Multi-UAV network assisted intelligent edge computing: Challenges and opportunities," *China Commun.*, vol. 19, no. 3, pp. 258–278, Mar. 2022.

[7] J. Plachy, Z. Becvar, E. C. Strinati, and N. Di Pietro, "Dynamic allocation of computing and communication resources in multi-access edge computing for mobile users," *IEEE T. Netw. Serv. Man.*, vol. 18, no. 2, pp. 2089–2106, Jun. 2021.

[8] S. Ko, K. Lee, H. Cho, Y. Hwang, and H. Jang, "Asynchronous federated learning with directed acyclic graph-based blockchain in edge computing: Overview, design, and challenges," *Exper. Syst. Appl.*, vol. 223, Aug. 2023, Art. no. 119896.

[9] M. Yan, R. Xiong, Y. Wang, and C. Li, "Edge computing task offloading optimization for a UAV-assisted Internet of Vehicles via deep reinforcement learning," *IEEE T. Veh. Technol.*, vol. 73, no. 4, pp. 5647–5658, Apr. 2024.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

[10] J. Huang, M. Zhang, J. Wan, Y. Chen, and N. Zhang, "Joint data caching and computation offloading in UAV-assisted Internet of Vehicles via federated deep reinforcement learning," *IEEE T. Veh. Technol.*, vol. 73, no. 11, pp. 17644–17656, Nov. 2024.

[11] X. Dai, Z. Xiao, H. Jiang, and J. C. Lui, "UAV-assisted task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2520–2534, Apr. 2023.

[12] Z. Yu, Y. Gong, S. Gong, and Y. Guo, "Joint task offloading and resource allocation in UAV-enabled mobile edge computing," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3147–3159, Apr. 2020.

[13] L. Wu, R. Zhang, Q. Li, C. Ma, and X. Shi, "A mobile edge computing-based applications execution framework for Internet of Vehicles," *Front. Comput. Sci-Chi.*, vol. 16, no. 5, Jan. 2022, Art. no. 165506.

[14] S. Raza et al., "An efficient task offloading scheme in vehicular edge computing," *J. Cloud Comp.*, vol. 9, pp. 1–14, Jun. 2020.

[15] W. Yang, Z. Liu, X. Liu, and Y. Ma, "Deep reinforcement learning-based low-latency task offloading for mobile-edge computing networks," *Appl. Soft Comput.*, vol. 166, Nov. 2024, Art. no. 112164.

[16] S. Zhu, L. Gui, D. Zhao, N. Cheng, Q. Zhang, and X. Lang, "Learning-based computation offloading approaches in UAVs-assisted edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 1, pp. 928–944, Jan. 2021.

[17] T. Zhang, Y. Wang, Y. Liu, W. Xu, and A. Nallanathan, "Cache-enabling UAV communications: Network deployment and resource allocation," *IEEE Trans. Wireless. Commun.*, vol. 19, no. 11, pp. 7470–7483, Nov. 2020.

[18] Q. Tang, B. Li, H. H. Yang, Y. Li, S. He, and K. Yang, "Delay and load fairness optimization with queuing model in multi-UAV assisted MEC: A deep reinforcement learning approach," *IEEE Trans. Netw. Serv. Man.*, vol. 22, no. 2, pp. 1247–1258, Dec. 2024.

[19] Z. Ning et al., "Intelligent edge computing in Internet of Vehicles: A joint computation offloading and caching solution," *IEEE Trans. Intell. Transp.*, vol. 22, no. 4, pp. 2212–2225, Apr. 2021.

[20] Y. Li, L. Li, and Z. Zhou, "Joint edge caching and computation offloading for heterogeneous tasks in MEC-enabled vehicular networks," *Veh. Commun.*, vol. 50, Dec. 2024, Art. no. 100860.

[21] L. Wang et al., "Joint task offloading and migration optimization in UAV-enabled dynamic MEC networks," *IEEE Trans. Service Comput.*, early access, Jun. 4, 2025, doi: 10.1109/TSC.2025.3576644.

[22] Y. Liu, P. Lin, M. Zhang, Z. Zhang, and F. R. Yu, "Mobile-aware service offloading for UAV-assisted IoV: A Multiagent tiny distributed learning approach," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21191–21201, Jun. 2024.

[23] S. Barick and C. Singhal, "UAV-assisted MEC architecture for collaborative task offloading in urban IoT environment," *IEEE Trans. Netw. Serv. Manag.*, vol. 22, no. 1, pp. 732–743, Feb. 2025.

[24] S. Shen, G. Shen, Z. Dai, K. Zhang, X. Kong, and J. Li, "Asynchronous federated deep-reinforcement-learning-based dependency task offloading for UAV-assisted vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31561–31574, Oct. 2024.

[25] L. Sun, L. Wan, J. Wang, L. Lin, and M. Gen, "Joint resource scheduling for UAV-enabled mobile edge computing system in Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15624–15632, Dec. 2023.

[26] F. Tian, J. Yang, S. Zhao, and M. Sawan, "NeuroCARE: A generic neuromorphic edge computing framework for healthcare applications," *Front. Neurosci*, vol. 17, Jan. 2023, Art. no. 1093865.

[27] T. Shi, T. Zhang, J. Loo, R. Huang, and Y. Wang, "Joint task offloading and channel allocation in spatial-temporal dynamic for MEC networks," *IEEE Trans. Ind. Informat.*, vol. 21, no. 8, pp. 6240–6250, Aug. 2025.

[28] K. Zheng, G. Jiang, X. Liu, K. Chi, X. Yao, and J. Liu, "DRL-based offloading for computation delay minimization in wireless-powered multi-access edge computing," *IEEE Trans. Commun.*, vol. 71, no. 3, pp. 1755–1770, Mar. 2023.

[29] W. Chu, X. Jia, Z. Yu, J. C. Lui, and Y. Lin, "Joint service caching, resource allocation and task offloading for MEC-based networks: A multi-layer optimization approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2958–2975, Apr. 2024.

[30] W. Wen, Y. Cui, T. Q. Quek, F.-C. Zheng, and S. Jin, "Joint optimal software caching, computation offloading and communications resource allocation for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7879–7894, Jul. 2020.

[31] G. Sun, L. Sheng, L. Luo, and H. Yu, "Game theoretic approach for multipriority data transmission in 5G vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 24672–24685, Dec. 2022.

[32] B. L. Nguyen, D. T. Ngo, N. H. Tran, M. N. Dao, and H. L. Vu, "Dynamic V2I/V2V cooperative scheme for connectivity and throughput enhancement," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 1236–1246, Feb. 2022.

[33] H. Guo, J. Liu, J. Ren, and Y. Zhang, "Intelligent task offloading in vehicular edge computing networks," *IEEE Trans. Wireless. Commun.*, vol. 27, no. 4, pp. 126–132, Aug. 2020.

[34] L. Liu, J. Feng, X. Mu, Q. Pei, D. Lan, and M. Xiao, "Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15513–15526, Dec. 2023.

[35] Q. Luo, C. Li, T. H. Luan, and W. Shi, "Collaborative data scheduling for vehicular edge computing via deep reinforcement learning," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9637–9650, Oct. 2020.

[36] B. Li, F. Chen, Z. Peng, P. Hou, and H. Ding, "Mobility-aware dynamic offloading strategy for C-V2X under multi-access edge computing," *Phys. Commun.*, vol. 49, Dec. 2021, Art. no. 101446.

[37] M. A. Mirza et al., "MCLA task offloading framework for 5G-NR-V2X-based heterogeneous VECNs," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 14329–14346, Dec. 2023.

[38] N. Fofana, A. B. Letaifa, and A. Rachedi, "Intelligent task offloading in vehicular networks: A deep reinforcement learning perspective," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 201–216, Jan. 2025.

[39] K. Gu, Z. Liu, and W. Jia, "Location-aware reliable task cooperative-computation scheme under fog computing-based IoVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 1, pp. 425–442, Jan. 2025.

[40] L. Zhang and N. Ansari, "Optimizing the operation cost for UAV-aided mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 6085–6093, Jun. 2021.

[41] L. Zhao, K. Yang, Z. Tan, X. Li, S. Sharma, and Z. Liu, "A novel cost optimization strategy for SDN-enabled UAV-assisted vehicular computation offloading," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3664–3674, Jun. 2021.

[42] S. Shen, H. Yang, K. Yang, K. Wang, and G. Zhang, "AoI-aware joint resource allocation in multi-UAV aided multi-access edge computing systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2596–2609, May/Jun. 2023.

[43] Z. Cheng, M. Min, Z. Gao, and L. Huang, "Joint task offloading and resource allocation for mobile edge computing in ultra-dense network," in *Proc. IEEE Global Commun. Conf.*, 2020, pp. 1–6.

[44] Y. Peng et al., "Data-empowered trajectory planning based on two-phase deep reinforcement learning method," *IEEE Internet Things J.*, early access, Feb. 24, 2025, doi: 10.1109/JIOT.2025.3545013.

[45] C. He et al., "An adaptive heuristic algorithm with a collaborative search framework for multi-UAV inspection planning," *Appl. Soft. Comput.*, vol. 174, Apr. 2025, Art. no. 112969.

[46] A. Ait Temghart, M. Marwan, and M. Baslam, "Optimization of computational offloading in the mobile edge: A game theoretic approach," *Cluster Comput.*, vol. 28, no. 3, pp. 1–15, 2025.

[47] F. Ahmad, X. Zhang, Z. Tang, F. Sabah, M. Azam, and R. Sarwar, "Deep deterministic policy gradients with a self-adaptive reward mechanism for image retrieval," *J. Supercomput.*, vol. 81, no. 1, p. 336, 2025.

[48] C. Li, Y. Zhang, X. Gao, and Y. Luo, "Energy-latency tradeoffs for edge caching and dynamic service migration based on DQN in mobile edge computing," *J. Parallel. Distrib. Comput.*, vol. 166, pp. 15–31, Aug. 2022.

[49] L. Zhang, S. Lai, J. Xia, C. Gao, D. Fan, and J. Ou, "Deep reinforcement learning based IRS-assisted mobile edge computing under physical-layer security," *Phys. Commun*, vol. 55, Dec. 2022, Art. no. 101896.

[50] B. Yamansavascilar, A. C. Baktir, C. Sonmez, A. Ozgovde, and C. Ersoy, "DeepEdge: A deep reinforcement learning based task orchestrator for edge computing," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 1, pp. 538–552, Jan./Feb. 2022.

[51] A. Mukhopadhyay, G. Iosifidis, and M. Ruffini, "Migration-aware network services with edge computing," *IEEE Trans. Netw. Service*, vol. 19, no. 2, pp. 1458–1471, Jun. 2022.

[52] Y. Liu et al., "Dependency-aware task scheduling in vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4961–4971, Jun. 2020.

[53] X. Wang and G. Chen, "Research on cache algorithm for internet of vehicles based on edge computing," in *Proc. 4th Int. Conf. Front. Technol. Inf. Comput. (ICFTIC)*, 2022, pp. 387–390.

[54] X. Chen and G. Liu, "Energy-efficient task offloading and resource allocation via deep reinforcement learning for augmented reality in mobile edge networks," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10843–10856, Jul. 2021.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.

[55] M. Yasir, S. K. uz Zaman, T. Maqsood, F. Rehman, and S. Mustafa, "CoPUP: Content popularity and user preferences aware content caching framework in mobile edge computing," *Cluster Comput.*, vol. 26, no. 1, pp. 267–281, 2023.
[56] C. Shang, Y. Huang, Y. Sun, and M. Guizani, "Joint computation offloading and service caching in mobile edge-cloud computing via deep reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 24, pp. 40331–40344, Dec. 2024.
[57] G. Wu, Z. Liu, M. Fan, and K. Wu, "Joint task offloading and resource allocation in multi-UAV multi-server systems: An attention-based deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 73, no. 8, pp. 11964–11978, Aug. 2024.
[58] Z. Xiao et al., "Multi-objective parallel task offloading and content caching in D2D-aided MEC networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 11, pp. 6599–6615, Nov. 2023.
[59] Y. Ren, X. Yu, X. Chen, S. Guo, and Q. Xue-Song, "Vehicular network edge intelligent management: A deep deterministic policy gradient approach for service offloading decision," in *Proc. Int. wireless Commun. Mobile Comput. (IWCMC)*, 2020, pp. 905–910.
[60] P. A. Apostolopoulos, G. Fragkos, E. E. Tsiropoulou, and S. Papavassiliou, "Data offloading in UAV-assisted multi-access edge computing systems under resource uncertainty," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 175–190, Jan. 2023.



Jingpan Bai received the Ph.D. degree in computer science and technology from Wuhan University of Technology, Wuhan, China, in 2022.

He is currently an Associate Professor with the School of Computer Science, Yangtze University, Jingzhou, China, also with the Hubei Key Laboratory of Oil and Gas Drilling and Production Engineering, Yangtze University, Wuhan, also with the Yunnan Key Laboratory of Service Computing, Yunnan University of Finance and Economics, Kunming, China, and also with the Yunnan Key Laboratory of Unmanned Autonomous Systems, Yunnan Minzu University, Kunming. His research interests include cloud computing, edge computing, big data processing, artificial intelligence, and related applications.



Jiahui Luo received the B.E. degree in computer science and technology from Guangxi Normal University for Nationalities, Chongzuo, China, in 2024. She is currently pursuing the M.E. degree in computer science and technology with Yangtze University, Jingzhou, China.

Her research interests focus on edge computing, artificial intelligence, and related applications.



Yuan Chen received the master's degree in software engineering from Northwest Minzu University, Lanzhou, China, in 2016.

She is currently a Lecturer with the School of Computer Science, Yangtze University, Jingzhou, China. Her research interests include image processing, edge computing, artificial intelligence, and related applications.



Yuming Tang received the bachelor's degree in IoT engineering from the School of Computer Science, Yangtze University, Jingzhou, China, in 2024, where he is currently pursuing the master's degree in computer science and technology.



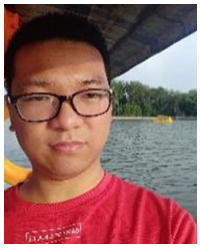
Li Jin received the B.E. degree in computer science and technology from Hubei University of Engineering New Technology College, Xiaogan, China, in 2024. He is currently pursuing the M.E. degree in computer technology with Yangtze University, Jingzhou, China.

His research interests mainly focus on edge computing, artificial intelligence, and applications related to the petroleum industry.



Yan Shi received the B.E. degree in Internet of Things Engineering from the School of Computer Science, Yangtze University, Jingzhou, China, in 2024, where she is currently pursuing the M.E. degree in computer science and technology.

Her research interests focus on edge computing, artificial intelligence, and related applications.



Bozhong Yang received the B.E. degree in computer science and technology from Furong College, Hunan University of Science and Arts, Changde, China, in 2022. He is currently pursuing the M.E. degree in computer science and technology with Yangtze University, Jingzhou, China.

His research interests focus on edge computing, artificial intelligence, and related applications.



Houling Ji received the Ph.D. degree in computer science and technology from Wuhan University, Wuhan, China, in 2020.

He is currently a Lecturer with the School of Computer Science, Yangtze University, Jingzhou, China. His research interests include real-time scheduling, data dissemination in mobile networks, Internet of Vehicles, deep learning, and related applications.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:34:16 UTC from IEEE Xplore. Restrictions apply.